

Dokumentacja: Android Studio

Przewodnik po tworzeniu aplikacji mobilnych w Android Studio. Każdy przykład kodu podany jest w dwóch wersjach: **Java** i **Kotlin**. Dokumentacja skupia się na praktycznych przykładach kodu, wyjaśnieniach plików projektu i budowaniu interfejsu użytkownika.

Spis treści

1. Tworzenie nowego projektu
2. Struktura plików projektu
3. Kluczowe pliki — wyjaśnienia i zawartość
4. Budowanie interfejsu — XML layouts
5. Układy (Layouts)
6. Widżety — podstawowe elementy UI
7. Widżety zaawansowane
8. Aktywność — łączenie XML z kodem
9. Obsługa zdarzeń
10. Praca z listami
11. Obrazy i zasoby graficzne
12. Kolory, style i motywy
13. Walidacja danych
14. Klasy pomocnicze i logika biznesowa
15. Dziedziczenie w Androidzie
16. Testy jednostkowe
17. Algorytmy — gotowe implementacje
18. Typowe wzorce aplikacji
19. Uruchamianie i emulacja
20. Najczęstsze błędy i rozwiązania

1. Tworzenie nowego projektu

1.1 Nowy projekt krok po kroku

1. Uruchom Android Studio → kliknij **New Project**
2. Wybierz **Empty Activity** → kliknij **Next**
3. Wypełnij pola:
 - **Name:** nazwa aplikacji (np. `GraWKosci`)
 - **Package name:** unikalny identyfikator (np. `com.example.grawkosci`)
 - **Language:** `Java` lub `Kotlin`
 - **Minimum SDK:** `API 21` (Android 5.0) — obsługuje większość urządzeń
4. Kliknij **Finish** — Gradle synchronizuje projekt (może potrwać minutę)

Uwaga: Jeśli Gradle zgłasza błąd synchronizacji, kliknij **File** → **Sync Project with Gradle Files**.

1.2 Tworzenie nowej aktywności

1. Prawym przyciskiem kliknij na pakiet w panelu Project
2. Wybierz **New** → **Activity** → **Empty Activity**
3. Podaj nazwę (np. `DrugaActivity`) → kliknij **Finish**

Android Studio automatycznie:

- Tworzy plik `DrugaActivity.java` / `DrugaActivity.kt`
- Tworzy `res/layout/activity_druga.xml`
- Dodaje wpis do `AndroidManifest.xml`

1.3 Tworzenie nowej klasy

1. Prawym przyciskiem na pakiet → **New** → **Java Class** (lub **Kotlin Class/File**)
2. Wpisz nazwę klasy (np. `Kosc`) → **Enter**

1.4 Dodawanie obrazów do projektu

1. Skopiuj pliki `.png` / `.jpg`
2. Prawym przyciskiem kliknij na `res/drawable` → **Paste**
3. Potwierdź operację

Ważne: Nazwy plików graficznych mogą zawierać **tylko małe litery, cyfry i podkreślenia** (np. `kosc1.png`, `tlo_aplikacji.jpg`). Wielkie litery i polskie znaki są niedozwolone.

1.5 Tworzenie folderu assets

1. Prawym przyciskiem na `app/src/main` → **New** → **Directory**
2. Wpisz `assets` → **Enter**
3. Umieść tam pliki tekstowe (np. `dane.txt`)

2. Struktura plików projektu

Po utworzeniu projektu Android Studio pokazuje następującą strukturę w widoku **Android**:

```
app/
├── manifests/
│   └── AndroidManifest.xml    ← Konfiguracja aplikacji (OBOWIĄZKOWY)
├── java/
│   └── com.example.mojaplikacja/
│       └── MainActivity.java  ← Kod głównej aktywności
└── res/
    ├── drawable/             ← Obrazy (.png, .jpg) i grafiki XML
    ├── layout/
    │   └── activity_main.xml  ← Układ ekranu głównego
    ├── values/
    │   ├── colors.xml        ← Definicje kolorów
    │   ├── strings.xml       ← Teksty aplikacji
    │   └── themes.xml        ← Motywy wyglądu
    └── mipmap/               ← Ikony aplikacji
```

Widok **Project** (zamiast **Android**) pokazuje fizyczną strukturę folderów — jest bardziej szczegółowy, ale mniej przejrzysty.

3. Kluczowe pliki — wyjaśnienia i zawartość

3.1 AndroidManifest.xml

Lokalizacja: `app/manifests/AndroidManifest.xml`

To najważniejszy plik konfiguracyjny. Deklaruje wszystkie aktywności aplikacji, uprawnienia i podstawowe metadane. Bez wpisu w manifeście aktywność nie uruchomi się.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.mojaplikacja">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.MojaAplikacja">

        <!-- Główna aktywność – ta, która uruchamia się po kliknięciu ikony -->
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- Każda kolejna aktywność MUSI być tutaj zadeklarowana -->
        <activity android:name=".DrugaActivity" />
        <activity android:name=".GaleriaActivity" />

    </application>

</manifest>
```

Kluczowe elementy:

- `package` — unikalny identyfikator aplikacji
- `android:label` — nazwa wyświetlana na urządzeniu
- `<activity android:name=".NazwaKlasy">` — rejestracja aktywności
- `<intent-filter>` z `MAIN` i `LAUNCHER` — oznacza aktywność startową

3.2 activity_main.xml (layout)

Lokalizacja: `res/layout/activity_main.xml`

Plik XML opisujący wygląd ekranu. Każda aktywność ma swój plik layout. Można go edytować w trybie **Code** (tekst XML) lub **Design** (wizualny przeciągnij-upuść). Zalecany jest tryb **Code** — daje pełną kontrolę.

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Korzeń layoutu – jeden element główny -->
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">
```

```

<!-- Każdy element ma obowiązkowe: id, layout_width, layout_height -->
<TextView
    android:id="@+id/tvTytuł"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Witaj!"
    android:textSize="24sp" />

<Button
    android:id="@+id/btnOK"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Kliknij mnie" />

</LinearLayout>

```

Najważniejsze atrybuty:

ATRYBUT	OPIS	PRZYKŁAD
android:id="@+id/..."	Identyfikator — do powiązania z kodem	@+id/btnOK
android:layout_width	Szerokość elementu	match_parent, wrap_content, 100dp
android:layout_height	Wysokość elementu	match_parent, wrap_content, 60dp
android:text	Wyświetlany tekst	"Kliknij mnie"
android:textSize	Rozmiar czcionki	16sp, 24sp
android:background	Kolor lub grafika tła	#FF5733, @color/red
android:padding	Odstęp wewnętrzny	8dp, 16dp
android:layout_margin	Odstęp zewnętrzny	10dp
android:gravity	Wyrównanie zawartości	center, start, end
android:visibility	Widoczność	visible, invisible, gone

Jednostki miar:

- dp** — niezależne od gęstości ekranu, **używaj do wymiarów i marginesów**
- sp** — skalowane przez ustawienia czcionki użytkownika, **używaj do tekstu**
- px** — piksele fizyczne, **unikaj**

3.3 MainActivity.java / MainActivity.kt

Lokalizacja: `java/com.example.../MainActivity.java`

Plik z logiką aktywności. Łączy pliki XML z kodem — pobiera elementy po ID i przypisuje im zachowanie.

Java:

```

package com.example.mojaplikacja;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

```

```

import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {

    // 1. Deklaruj pola dla elementów UI jako pola klasy
    private TextView tvWynik;
    private Button btnOK;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // 2. Wskaż plik XML z layoutem aktywności
        setContentView(R.layout.activity_main);

        // 3. Pobierz referencje do elementów XML po ich ID
        tvWynik = findViewById(R.id.tvWynik);
        btnOK = findViewById(R.id.btnOK);

        // 4. Przypisz zachowania (zdarzenia)
        btnOK.setOnClickListener(v → {
            tvWynik.setText("Kliknięto przycisk!");
        });
    }
}

```

Kotlin:

```

package com.example.mojaplikacja

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.TextView

class MainActivity : AppCompatActivity() {

    // lateinit = inicjalizacja później (w onCreate)
    private lateinit var tvWynik: TextView
    private lateinit var btnOK: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        tvWynik = findViewById(R.id.tvWynik)
        btnOK = findViewById(R.id.btnOK)

        btnOK.setOnClickListener {
            tvWynik.text = "Kliknięto przycisk!"
        }
    }
}

```

```
}  
  
}
```

Ważne: Kolejność w `onCreate` jest obowiązkowa:

1. `super.onCreate(savedInstanceState)` — zawsze pierwsze
2. `setContentView(R.layout.activity_main)` — załaduj XML
3. `findViewById(...)` — po `setContentView`, inaczej `NullPointerException`

3.4 colors.xml

Lokalizacja: `res/values/colors.xml`

Definicje kolorów używanych w całej aplikacji. Zamiast wpisywać hex bezpośrednio w XML, definiuje się tu nazwane kolory.

```
<?xml version="1.0" encoding="utf-8"?>  
<resources>  
    <color name="colorPrimary">#D2691E</color>  
    <color name="colorBackground">#F5F5DC</color>  
    <color name="colorText">#333333</color>  
    <color name="colorWhite">#FFFFFF</color>  
    <color name="colorError">#DC143C</color>  
    <color name="colorSuccess">#4CAF50</color>  
  
    <!-- Kolor z kanałem alpha: #AARRGGBB -->  
    <!-- AA = przezroczystość: 00=pełna, FF=brak -->  
    <color name="tloKosci">#ED27C121</color>  
    <color name="tloRzut">#ED275021</color>  
</resources>
```

Użycie w XML:

```
android:background="@color/colorBackground"  
android:textColor="@color/colorText"
```

Użycie w kodzie Java:

```
int kolor = ContextCompat.getColor(this, R.color.colorPrimary);  
textView.setTextColor(kolor);
```

Użycie w kodzie Kotlin:

```
val kolor = ContextCompat.getColor(this, R.color.colorPrimary)  
textView.setTextColor(kolor)
```

3.5 strings.xml

Lokalizacja: `res/values/strings.xml`

Teksty aplikacji. Używanie tego pliku zamiast wpisywania tekstu bezpośrednio ułatwia późniejsze tłumaczenie.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Moja Aplikacja</string>
    <string name="btn_rzut">RZUT</string>
    <string name="btn_resetuj">RESETUJ</string>
    <string name="wynik_prefix">Wynik: </string>
    <string name="blad_puste_pole">Pole nie może być puste</string>
</resources>
```

Użycie w XML:

```
android:text="@string/btn_rzut"
```

Użycie w kodzie:

```
// Java
String tekst = getString(R.string.wynik_prefix);

// Kotlin
val tekst = getString(R.string.wynik_prefix)
```

3.6 build.gradle (Module: app)

Lokalizacja: `Gradle Scripts/build.gradle (Module: app)`

Konfiguracja projektu i zależności (bibliotek). Edytuj go gdy chcesz dodać zewnętrzne biblioteki.

```
plugins {
    id 'com.android.application'
}

android {
    compileSdk 34

    defaultConfig {
        applicationId "com.example.mojaplikacja"
        minSdk 21          // Minimalna wersja Androida (API 21 = Android 5.0)
        targetSdk 34       // Wersja docelowa
        versionCode 1
        versionName "1.0"
    }
}

dependencies {
    // Standardowe biblioteki AndroidX
    implementation 'androidx.appcompat:appcompat:1.6.1'
    implementation 'com.google.android.material:material:1.11.0'
```

```
implementation 'androidx.constraintlayout:constraintlayout:2.1.4'

// Biblioteki do testów jednostkowych
testImplementation 'junit:junit:4.13.2'
androidTestImplementation 'androidx.test.ext:junit:1.1.5'
}
```

Po edycji kliknij **Sync Now** w górnym pasku lub **File → Sync Project with Gradle Files**.

4. Budowanie interfejsu — XML layouts

4.1 Jak działa plik XML layout

Każdy ekran aplikacji ma odpowiadający mu plik XML w `res/layout/`. Nazwy plików są powiązane z aktywnością:

- `MainActivity.java` → `activity_main.xml`
- `GaleriaActivity.java` → `activity_galeria.xml`

Powiązanie następuje przez wywołanie w kodzie:

```
setContentView(R.layout.activity_main); // Java
setContentView(R.layout.activity_galeria) // Kotlin
```

4.2 Przestrzenie nazw XML

Każdy plik layout musi zawierać deklarację przestrzeni nazw w elemencie głównym:

```
xmlns:android="http://schemas.android.com/apk/res/android"
```

Opcjonalnie:

```
xmlns:app="http://schemas.android.com/apk/res-auto"    <!-- dla ConstraintLayout -->
xmlns:tools="http://schemas.android.com/tools"         <!-- tylko w edytorze, nie w apce -->
```

4.3 Odwołania do zasobów w XML

```
<!-- Kolor z colors.xml -->
android:background="@color/colorBackground"
```

```
<!-- Kolor bezpośredni -->
android:background="#F5F5DC"
```

```
<!-- Tekst z strings.xml -->
android:text="@string/btn_rzut"
```

```
<!-- Tekst bezpośredni -->
android:text="RZUT"
```

```
<!-- Obraz z res/drawable/ -->
android:src="@drawable/kosc1"
```



```
←!— Identyfikator elementu →  
android:id="@+id/tvWynik"    ←!— @+ = utwórz nowe ID →  
  
←!— Odwołanie do istniejącego ID →  
app:layout_constraintTop_toBottomOf="@id/tvTytuł"  ←!— @id, bez + →
```

5. Układy (Layouts)

Układy to kontenery definiujące sposób rozmieszczenia elementów. Jeden layout może być zagnieżdżony w innym.

5.1 LinearLayout — układ liniowy

Elementy ułożone jeden po drugim — pionowo lub poziomo.

Pionowy (vertical) — elementy jeden pod drugim:

```
<LinearLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:orientation="vertical"  
    android:padding="16dp"  
    android:background="#F5F5DC">  
  
    <TextView  
        android:id="@+id/tvTytuł"  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:text="Tytuł"  
        android:textSize="24sp"  
        android:textColor="#A52A2A"  
        android:gravity="center"  
        android:layout_marginBottom="16dp" />  
  
    <EditText  
        android:id="@+id/etImie"  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:hint="Wpisz imię"  
        android:layout_marginBottom="12dp" />  
  
    <Button  
        android:id="@+id/btnZatwierdz"  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="ZATWIERDŹ"  
        android:layout_gravity="center" />  
  
</LinearLayout>
```

Poziomy (horizontal) — elementy obok siebie:

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <ImageView
        android:id="@+id/ivKosc1"
        android:layout_width="0dp"
        android:layout_height="60dp"
        android:layout_weight="1"
        android:src="@drawable/kosc0"
        android:layout_margin="9dp" />

    <ImageView
        android:id="@+id/ivKosc2"
        android:layout_width="0dp"
        android:layout_height="60dp"
        android:layout_weight="1"
        android:src="@drawable/kosc0"
        android:layout_margin="9dp" />

    <ImageView
        android:id="@+id/ivKosc3"
        android:layout_width="0dp"
        android:layout_height="60dp"
        android:layout_weight="1"
        android:src="@drawable/kosc0"
        android:layout_margin="9dp" />

</LinearLayout>

```

layout_weight — proporcjonalne zajmowanie miejsca:

- Gdy `layout_width="0dp"` i `layout_weight="1"` — element zajmuje równą część przestrzeni
- `layout_weight="2"` vs `layout_weight="1"` → pierwszy zajmie 2/3, drugi 1/3

5.2 ScrollView — przewijanie

Gdy zawartość może być dłuższa niż ekran:

```

<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- ScrollView może mieć TYLKO JEDEN bezpośredni element potomny -->

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <!-- Wiele elementów formularza -->

```

```

        <EditText android:id="@+id/etImie" ... />
        <EditText android:id="@+id/etNazwisko" ... />
        <EditText android:id="@+id/etEmail" ... />
        <EditText android:id="@+id/etTelefon" ... />
        <EditText android:id="@+id/etAdres" ... />
        <Button android:id="@+id/btnWyslij" ... />

    </LinearLayout>

</ScrollView>

```

5.3 ConstraintLayout — układ z ograniczeniami

Pozycjonowanie elementów względem siebie lub krawędzi ekranu. Domyślny layout w nowych projektach.

```

<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/tvNaglowek"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Nagłówek"
        android:textSize="22sp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="32dp" />

    <Button
        android:id="@+id/btnOK"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="OK"
        app:layout_constraintTop_toBottomOf="@id/tvNaglowek"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="16dp" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Rodzaje ograniczeń (constraints):

app:layout_constraintTop_toTopOf="parent"	←!— góra elementu = góra rodzica →
app:layout_constraintTop_toBottomOf="@id/inny"	←!— góra elementu = dół innego →
app:layout_constraintBottom_toBottomOf="parent"	←!— dół elementu = dół rodzica →
app:layout_constraintStart_toStartOf="parent"	←!— lewa krawędź = lewa rodzica →
app:layout_constraintEnd_toEndOf="parent"	←!— prawa krawędź = prawa rodzica →

5.4 Zagnieżdżanie układów

←!— Pionowy układ główny →

```
<LinearLayout
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#ED27C121">

    <!-- Tytuł -->
    <TextView
        android:text="Gra w kości"
        android:textSize="28sp"
        android:textColor="#FFFFFF"
        android:gravity="center"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="10dp" />
```

←!— ZAGNIEŻDŻONY poziomy układ dla 5 kości →

```
<LinearLayout
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="#FFFFFF">

    <ImageView android:id="@+id/ivKosc1"
        android:layout_width="0dp" android:layout_height="60dp"
        android:layout_weight="1" android:layout_margin="9dp"
        android:src="@drawable/kosc0" />
    <ImageView android:id="@+id/ivKosc2"
        android:layout_width="0dp" android:layout_height="60dp"
        android:layout_weight="1" android:layout_margin="9dp"
        android:src="@drawable/kosc0" />
    <ImageView android:id="@+id/ivKosc3"
        android:layout_width="0dp" android:layout_height="60dp"
        android:layout_weight="1" android:layout_margin="9dp"
        android:src="@drawable/kosc0" />
    <ImageView android:id="@+id/ivKosc4"
        android:layout_width="0dp" android:layout_height="60dp"
        android:layout_weight="1" android:layout_margin="9dp"
        android:src="@drawable/kosc0" />
    <ImageView android:id="@+id/ivKosc5"
        android:layout_width="0dp" android:layout_height="60dp"
        android:layout_weight="1" android:layout_margin="9dp"
        android:src="@drawable/kosc0" />

</LinearLayout>

<!-- Wynik -->
<TextView
    android:id="@+id/tvWynik"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
```

```
        android:text="0"
        android:textSize="40sp"
        android:textColor="#FFFFFF"
        android:gravity="center"
        android:layout_margin="10dp" />
```

←!— Przycisk —>

```
<Button
    android:id="@+id/btnRzut"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="RZUT"
    android:backgroundTint="#ED275021"
    android:textColor="#FFFFFF"
    android:textSize="18sp"
    android:layout_margin="10dp" />
```

```
</LinearLayout>
```

6. Widżety — podstawowe elementy UI

6.1 TextView — etykieta tekstowa

XML:

```
<TextView
    android:id="@+id/tvNapis"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Przykładowy tekst"
    android:textSize="18sp"
    android:textColor="#333333"
    android:textStyle="bold"
    android:gravity="center"
    android:background="#E8E8E8"
    android:padding="8dp"
    android:layout_margin="8dp" />
```

Java — zmiana tekstu programowo:

```
TextView tv = findViewById(R.id.tvNapis);

tv.setText("Nowy tekst");
tv.setText("Wynik: " + liczba);
tv.setText(String.valueOf(42));

// Zmiana koloru
tv.setTextColor(Color.RED);
tv.setTextColor(Color.parseColor("#4CAF50"));
tv.setTextColor(ContextCompat.getColor(this, R.color.colorSuccess));
```

```
// Zmiana rozmiaru
tv.setTextSize(20); // w SP

// Widoczność
tv.setVisibility(View.VISIBLE); // widoczny
tv.setVisibility(View.INVISIBLE); // niewidoczny, zajmuje miejsce
tv.setVisibility(View.GONE); // niewidoczny, nie zajmuje miejsca
```

Kotlin:

```
val tv = findViewById<TextView>(R.id.tvNapis)

tv.text = "Nowy tekst"
tv.text = "Wynik: $liczba"

tv.setTextColor(Color.RED)
tv.setTextColor(Color.parseColor("#4CAF50"))

tv.visibility = View.VISIBLE
tv.visibility = View.GONE
```

6.2 EditText — pole tekstowe wejściowe

XML:

```
<EditText
    android:id="@+id/etEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Wpisz adres e-mail"
    android:inputType="textEmailAddress"
    android:textSize="16sp"
    android:padding="12dp"
    android:layout_margin="8dp" />
```

Typy wejściowe (`inputType`):

WARTOŚĆ	OPIS
<code>text</code>	Zwykły tekst
<code>textPassword</code>	Hasło (znaki maskowane)
<code>textEmailAddress</code>	E-mail (klawiatura z @)
<code>number</code>	Tylko cyfry bez znaku
<code>numberSigned</code>	Cyfry ze znakiem (ujemne)
<code>numberDecimal</code>	Liczby z przecinkiem
<code>textMultiLine</code>	Wiele linii tekstu
<code>phone</code>	Numer telefonu

Java — pobieranie i ustawianie tekstu:

```
EditText et = findViewById(R.id.etEmail);

// Pobranie tekstu (zawsze używaj trim() do usunięcia spacji)
String tekst = et.getText().toString().trim();

// Ustawienie tekstu
et.setText("jan@example.com");

// Wyczyszczenie pola
et.setText("");

// Ustawienie błędu (czerwony napis pod polem)
et.setError("Nieprawidłowy e-mail");
```

Kotlin:

```
val et = findViewById<EditText>(R.id.etEmail)

val tekst = et.text.toString().trim()
et.setText("jan@example.com")
et.setText("")
et.error = "Nieprawidłowy e-mail"
```

6.3 Button — przycisk

XML:

```
<Button
    android:id="@+id/btnZatwierdz"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="ZATWIERDŹ"
    android:textSize="16sp"
    android:textColor="#FFFFFF"
```

```
android:backgroundTint="#D2691E"
android:layout_gravity="center"
android:layout_margin="10dp"
android:padding="12dp" />
```

Java — obsługa kliknięcia:

```
Button btn = findViewById(R.id.btnZatwierdz);

// Lambda (Java 8+) – zalecane
btn.setOnClickListener(v → {
    // kod wykonywany po kliknięciu
    tvWynik.setText("Kliknięto!");
});

// Anonimowa klasa (stary styl)
btn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        tvWynik.setText("Kliknięto!");
    }
});
```

Kotlin:

```
val btn = findViewById<Button>(R.id.btnZatwierdz)

btn.setOnClickListener {
    tvWynik.text = "Kliknięto!"
}
```

Zmiana tekstu przycisku programowo:

```
// Java
btn.setText("Wyłącz");

// Kotlin
btn.text = "Wyłącz"
```

6.4 ImageView — obraz

XML:

```
<ImageView
    android:id="@+id/ivKosc"
    android:layout_width="60dp"
    android:layout_height="60dp"
    android:src="@drawable/kosc0"
```



```
android:scaleType="fitCenter"
android:layout_margin="9dp" />
```

scaleType — sposób skalowania obrazu:

- **fitCenter** — dopasuj zachowując proporcje, wyśrodkuj (*zalecane*)
- **centerCrop** — wypełnij przycinając
- **fitXY** — rozciągnij do pełnego wypełnienia (bez proporcji)
- **center** — wyśrodkuj bez skalowania

Java — ustawianie obrazu programowo:

```
ImageView iv = findViewById(R.id.ivKosc);

// Ustaw konkretny zasób
iv.setImageResource(R.drawable.kosc3);

// Ustaw zasób po nazwie (dynamicznie – np. "kosc" + liczba)
String nazwa = "kosc" + liczba0czek; // np. "kosc3"
int resId = getResources().getIdentifier(nazwa, "drawable", getPackageName());
iv.setImageResource(resId);

// Przezroczystość: 0.0f = całkowicie przezroczysty, 1.0f = nieprzezroczysty
iv.setAlpha(0.5f); // 50% przezroczysty
iv.setAlpha(1.0f); // pełna widoczność
```

Kotlin:

```
val iv = findViewById<ImageView>(R.id.ivKosc)

iv.setImageResource(R.drawable.kosc3)

val nazwa = "kosc$liczba0czek"
val resId = resources.getIdentifier(nazwa, "drawable", packageName)
iv.setImageResource(resId)

iv.alpha = 0.5f
iv.alpha = 1.0f
```

6.5 RadioButton i RadioGroup

RadioGroup zapewnia, że tylko jeden **RadioButton** jest zaznaczony na raz.

XML:

```
<RadioGroup
    android:id="@+id/rgTypPrzesylki"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">
```

```

<RadioButton
    android:id="@+id/rbPocztowka"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Pocztówka – 2,50 zł"
    android:textSize="16sp" />

<RadioButton
    android:id="@+id/rbList"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="List – 5,00 zł"
    android:textSize="16sp" />

<RadioButton
    android:id="@+id/rbPaczka"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Paczka – 15,00 zł"
    android:textSize="16sp"
    android:checked="true" />

</RadioGroup>

```

Java — obsługa wyboru:

```

RadioGroup rg = findViewById(R.id.rgTypPrzesylki);

// Sprawdzenie wybranej opcji po kliknięciu przycisku
int zaznaczoneId = rg.getCheckedRadioButtonId();

if (zaznaczoneId == -1) {
    Toast.makeText(this, "Wybierz opcję!", Toast.LENGTH_SHORT).show();
    return;
}

String cena;
if (zaznaczoneId == R.id.rbPocztowka) {
    cena = "2,50 zł";
} else if (zaznaczoneId == R.id.rbList) {
    cena = "5,00 zł";
} else {
    cena = "15,00 zł";
}

tvCena.setText("Cena: " + cena);

// Reakcja na KAŻDĄ zmianę wyboru
rg.setOnCheckedChangeListener((group, checkedId) -> {
    if (checkedId == R.id.rbPocztowka) {
        tvCena.setText("Cena: 2,50 zł");
    } else if (checkedId == R.id.rbList) {
        tvCena.setText("Cena: 5,00 zł");
    } else {

```

```

        tvCena.setText("Cena: 15,00 zł");
    }
});

```

Kotlin:

```

val rg = findViewById<RadioGroup>(R.id.rgTypPrzesylki)

val zaznaczoneId = rg.checkedRadioButtonId
if (zaznaczoneId == -1) {
    Toast.makeText(this, "Wybierz opcję!", Toast.LENGTH_SHORT).show()
    return
}

val cena = when (zaznaczoneId) {
    R.id.rbPocztowka → "2,50 zł"
    R.id.rbList      → "5,00 zł"
    else             → "15,00 zł"
}

tvCena.text = "Cena: $cena"

```

6.6 SeekBar — suwak

XML:

```

<SeekBar
    android:id="@+id/sbRozmiar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:min="8"
    android:max="60"
    android:progress="20"
    android:layout_margin="16dp" />

```

Java — reagowanie na zmianę w czasie rzeczywistym:

```

SeekBar sb = findViewById(R.id.sbRozmiar);
TextView tvLabel = findViewById(R.id.tvLabel);
TextView tvProba = findViewById(R.id.tvProba);

sb.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
    @Override
    public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
        // Wywołane przy każdej zmianie – tutaj aktualizuj UI
        tvLabel.setText("Rozmiar: " + progress);
        tvProba.setTextSize(progress); // Zmień rozmiar czcionki
    }

    @Override

```

```

    public void onStartTrackingTouch(SeekBar seekBar) {
        // Użytkownik zaczął przesuwac
    }

    @Override
    public void onStopTrackingTouch(SeekBar seekBar) {
        // Użytkownik skończył przesuwac
    }
});

// Pobieranie aktualnej wartości
int wartość = sb.getProgress();

```

Kotlin:

```

val sb = findViewById<SeekBar>(R.id.sbRozmiar)

sb.setOnSeekBarChangeListener(object : SeekBar.OnSeekBarChangeListener {
    override fun onProgressChanged(seekBar: SeekBar, progress: Int, fromUser: Boolean) {
        tvLabel.text = "Rozmiar: $progress"
        tvProba.setTextSize(progress.toFloat())
    }
    override fun onStartTrackingTouch(seekBar: SeekBar) {}
    override fun onStopTrackingTouch(seekBar: SeekBar) {}
})

```

6.7 Spinner — lista rozwijana

XML:

```

<Spinner
    android:id="@+id/spinnerDzial"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp" />

```

Java — konfiguracja i obsługa:

```

Spinner spinner = findViewById(R.id.spinnerDzial);

// Dane dla spinnera
String[] opcje = {"IT", "HR", "Finanse", "Marketing", "Logistyka"};

// Adapter
ArrayAdapter<String> adapter = new ArrayAdapter<>(
    this,
    android.R.layout.simple_spinner_item,
    opcje
);

```

```

adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spinner.setAdapter(adapter);

// Obsługa wyboru
spinner.setOnItemClickListener(new AdapterView.OnItemClickListener() {

    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
        String wybrany = parent.getItemAtPosition(position).toString();
        tvWynik.setText("Wybrano: " + wybrany);
    }

    @Override
    public void onNothingSelected(AdapterView<?> parent) {}
});

// Pobranie aktualnie wybranej pozycji
int pozycja = spinner.getSelectedItemId();
String wartość = spinner.getSelectedItem().toString();

```

Kotlin:

```

val spinner = findViewById<Spinner>(R.id.spinnerDzial)
val opcje = arrayOf("IT", "HR", "Finanse", "Marketing")

val adapter = ArrayAdapter<String>(this, android.R.layout.simple_spinner_item, opcje)
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
spinner.adapter = adapter

spinner.setOnItemClickListener { parent, view, pos, id: Long } -> {
    override fun onItemClick(parent: AdapterView<*>, view: View?, pos: Int, id: Long) {
        val wybrany = parent.getItemAtPosition(pos).toString()
        tvWynik.text = "Wybrano: $wybrany"
    }

    override fun onNothingSelected(parent: AdapterView<*>) {}
}

```

Alternatywnie: zdefiniuj opcje bezpośrednio w `res/values/arrays.xml`:

```

<!-- res/values/arrays.xml -->
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="dzialy">
        <item>IT</item>
        <item>HR</item>
        <item>Finanse</item>
    </string-array>
</resources>

```

←!— W layout XML – automatyczne podłączenie →

```
<Spinner
    android:id="@+id/spinnerDzial"
    android:entries="@array/działy"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

6.8 CheckBox — pole wyboru

XML:

```
<CheckBox
    android:id="@+id/cbZgoda"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Akceptuję regulamin"
    android:textSize="14sp"
    android:checked="false" />
```

Java:

```
CheckBox cb = findViewById(R.id.cbZgoda);

// Sprawdzenie czy zaznaczony
if (cb.isChecked()) {
    tvWynik.setText("Zaakceptowano regulamin");
} else {
    tvWynik.setText("Brak akceptacji");
}

// Reakcja na zmianę
cb.setOnCheckedChangeListener((buttonView, isChecked) → {
    if (isChecked) {
        btnZatwierdz.setEnabled(true);
    } else {
        btnZatwierdz.setEnabled(false);
    }
});
```

Kotlin:

```
val cb = findViewById<CheckBox>(R.id.cbZgoda)

if (cb.isChecked) {
    tvWynik.text = "Zaakceptowano"
}

cb.setOnCheckedChangeListener { _, isChecked →
```

```
        btnZatwierdz.isEnabled = isChecked
    }
}
```

6.9 ListView — lista przewijalna

XML:

```
<ListView
    android:id="@+id/lvNotatki"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:divider="#DC143C"
    android:dividerHeight="1dp" />
```

Java — pełna konfiguracja z dynamicznym dodawaniem:

```
ListView lv = findViewById(R.id.lvNotatki);

// Lista danych
ArrayList<String> lista = new ArrayList<>();
lista.add("Notatka 1");
lista.add("Notatka 2");
lista.add("Notatka 3");

// Adapter łączy dane z ListView
ArrayAdapter<String> adapter = new ArrayAdapter<>(
    this,
    android.R.layout.simple_list_item_1, // wbudowany layout elementu
    lista
);
lv.setAdapter(adapter);

// Dodawanie nowego elementu
String nowyElement = "Nowa notatka";
lista.add(nowyElement);
adapter.notifyDataSetChanged(); // OBOWIĄZKOWE — powiadamia listę o zmianach

// Kliknięcie na element listy
lv.setOnItemClickListener((parent, view, position, id) → {
    String wybrany = lista.get(position);
    Toast.makeText(this, "Wybrano: " + wybrany, Toast.LENGTH_SHORT).show();
});
```

Kotlin:

```
val lv = findViewById<ListView>(R.id.lvNotatki)
val lista = ArrayList<String>()
lista.add("Notatka 1")
```

```

lista.add("Notatka 2")

val adapter = ArrayAdapter<this, android.R.layout.simple_list_item_1, lista>
lv.adapter = adapter

// Dodawanie
lista.add("Nowa notatka")
adapter.notifyDataSetChanged()

lv.setOnItemClickListener { _, _, position, _ ->
    Toast.makeText<this, "Wybrano: ${lista[position]}", Toast.LENGTH_SHORT>.show()
}

```

7. Widżety zaawansowane

7.1 RecyclerView — nowoczesna lista

Wydajniejsza alternatywa dla ListView. Wymaga adaptera i ViewHolder.

Dodaj do `build.gradle` (dependencies):

```
implementation 'androidx.recyclerview:recyclerview:1.3.0'
```

XML:

```

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1" />

```

Layout jednego elementu listy — utwórz `res/layout/item_element.xml`:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="12dp">

    <TextView
        android:id="@+id/tvTresc"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="16sp" />

</LinearLayout>

```

Adapter (Java):


```

import androidx.recyclerview.widget.RecyclerView;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import java.util.ArrayList;

public class MojAdapter extends RecyclerView.Adapter<MojAdapter.ViewHolder> {

    private ArrayList<String> dane;

    public MojAdapter(ArrayList<String> dane) {
        this.dane = dane;
    }

    // ViewHolder przechowuje referencje do widoków jednego elementu
    public static class ViewHolder extends RecyclerView.ViewHolder {
        TextView tvTresc;

        public ViewHolder(View itemView) {
            super(itemView);
            tvTresc = itemView.findViewById(R.id.tvTresc);
        }
    }

    @Override
    public ViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_element, parent, false);
        return new ViewHolder(view);
    }

    @Override
    public void onBindViewHolder(ViewHolder holder, int position) {
        holder.tvTresc.setText(dane.get(position));
    }

    @Override
    public int getItemCount() {
        return dane.size();
    }

    public void dodaj(String element) {
        dane.add(element);
        notifyItemInserted(dane.size() - 1);
    }
}

```

Inicjalizacja w Activity (Java):

```

RecyclerView rv = findViewById(R.id.recyclerView);
ArrayList<String> lista = new ArrayList<>();

```

```
lista.add("Element 1");

MojAdapter adapter = new MojAdapter(lista);
rv.setLayoutManager(new LinearLayoutManager(this));
rv.setAdapter(adapter);

// Dodawanie
adapter.dodaj("Nowy element");
```

Kotlin:

```
val rv = findViewById<RecyclerView>(R.id.recyclerView)
val lista = ArrayList<String>()

val adapter = MojAdapter(lista)
rv.layoutManager = LinearLayoutManager(this)
rv.adapter = adapter
```

7.2 AlertDialog — okno dialogowe

Java:

```
// Prosty dialog potwierdzenia
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Potwierdzenie");
builder.setMessage("Czy chcesz usunąć element?");

builder.setPositiveButton("TAK", (dialog, which) → {
    // Akcja po kliknięciu TAK
    usunElement();
});

builder.setNegativeButton("NIE", (dialog, which) → {
    dialog.dismiss(); // Zamknij dialog
});

builder.show();
```

Dialog z polem EditText (Java):

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Podaj wartość");

final EditText et = new EditText(this);
et.setInputType(InputType.TYPE_CLASS_NUMBER);
builder.setView(et);

builder.setPositiveButton("OK", (dialog, which) → {
    String wartość = et.getText().toString().trim();
```

```
tvWynik.setText("Wpisałeś: " + wartość);
});

builder.setNegativeButton("Anuluj", null);
builder.show();
```

Kotlin:

```
AlertDialog.Builder(this)
    .setTitle("Potwierdzenie")
    .setMessage("Czy chcesz usunąć element?")
    .setPositiveButton("TAK") { _, _ → usunElement() }
    .setNegativeButton("NIE") { dialog, _ → dialog.dismiss() }
    .show()
```

7.3 CardView — karty

Dodaj do `build.gradle` :

```
implementation 'androidx.cardview:cardview:1.0.0'
```

XML:

```
<androidx.cardview.widget.CardView
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="4dp"
    app:cardBackgroundColor="#FFFFFF">

    <LinearLayout
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp">

        <TextView
            android:id="@+id/tvTytułKarty"
            android:text="Tytuł karty"
            android:textSize="18sp"
            android:textStyle="bold"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />

        <TextView
            android:id="@+id/tvTrescKarty"
            android:text="Zawartość karty"
```

```

        android:textSize="14sp"
        android:layout_marginTop="8dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    </LinearLayout>

</androidx.cardview.widget.CardView>

```

7.4 DatePicker i TimePicker — wybieranie daty i czasu

XML:

```

<DatePicker
    android:id="@+id/datePicker"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />

<TimePicker
    android:id="@+id/timePicker"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:timePickerMode="spinner" />

```

Java — pobieranie wybranej daty i czasu:

```

DatePicker dp = findViewById(R.id.datePicker);
TimePicker tp = findViewById(R.id.timePicker);

// Data
int rok    = dp.getYear();
int miesiac = dp.getMonth() + 1; // UWAGA: miesiące są od 0, więc +1
int dzien  = dp.getDayOfMonth();
String data = dzien + "." + miesiac + "." + rok;

// Czas
int godzina = tp.getHour();
int minuta  = tp.getMinute();
String czas = String.format("%02d:%02d", godzina, minuta);

tvWynik.setText("Data: " + data + "   Czas: " + czas);

```

Kotlin:

```

val dp = findViewById<DatePicker>(R.id.datePicker)
val tp = findViewById<TimePicker>(R.id.timePicker)

val data = "${dp.dayOfMonth}.${dp.month + 1}.${dp.year}"
val czas = "%02d:%02d".format(tp.hour, tp.minute)

```

```
tvWynik.text = "Data: $data  Czas: $czas"
```

8. Aktywność — łączenie XML z kodem

8.1 Cykl życia aktywności

Aktywność przechodzi przez stany podczas pracy. Najważniejsza jest metoda `onCreate()`.



Metody cyklu życia:

METODA	KIEDY WYWOŁYWANA	TYPOWE ZASTOSOWANIE
<code>onCreate()</code>	Tworzenie aktywności	Inicjalizacja UI, <code>setContentView()</code> , <code>findViewById()</code>
<code>onStart()</code>	Aktywność staje się widoczna	Rzadko używana
<code>onResume()</code>	Aktywność aktywna	Wznowienie animacji, timerów
<code>onPause()</code>	Inna aktywność na pierwszym planie	Pauzowanie animacji
<code>onStop()</code>	Aktywność niewidoczna	Zapis danych
<code>onDestroy()</code>	Aktywność niszczona	Zwalnianie zasobów

8.2 Pełny szablon aktywności

Java:

```
package com.example.mojaplikacja;

import androidx.appcompat.app.AppCompatActivity;
import android.graphics.Color;
import android.os.Bundle;
import android.util.Log;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.SeekBar;
import android.widget.TextView;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {

    // Stała do logowania
    private static final String TAG = "MainActivity";

    // Pola UI – deklaruj jako pola klasy
    private TextView tvWynik;
    private EditText etWejscie;
```

```

private Button btnAkcja;
private ImageView ivObraz;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main); // MUSI być przed findViewById

    // Inicjalizacja widoków
    tvWynik = findViewById(R.id.tvWynik);
    etWejscie = findViewById(R.id.etWejscie);
    btnAkcja = findViewById(R.id.btnAkcja);
    ivObraz = findViewById(R.id.ivObraz);

    // Inicjalizacja zdarzeń
    btnAkcja.setOnClickListener(v → wykonajAkcje());

    Log.d(TAG, "Aplikacja uruchomiona");
}

private void wykonajAkcje() {
    String wejscie = etWejscie.getText().toString().trim();

    if (wejscie.isEmpty()) {
        Toast.makeText(this, "Pole nie może być puste", Toast.LENGTH_SHORT).show();
        return;
    }

    tvWynik.setText("Wynik: " + wejscie);
}
}

```

Kotlin:

```

package com.example.mojaplikacja

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.util.Log
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast

class MainActivity : AppCompatActivity() {

    private val TAG = "MainActivity"

    private lateinit var tvWynik: TextView
    private lateinit var etWejscie: EditText
    private lateinit var btnAkcja: Button

    override fun onCreate(savedInstanceState: Bundle?) {

```

```

        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        tvWynik = findViewById(R.id.tvWynik)
        etWejscie = findViewById(R.id.etWejscie)
        btnAkcja = findViewById(R.id.btnAkcja)

        btnAkcja.setOnClickListener { wykonajAkcje() }

        Log.d(TAG, "Aplikacja uruchomiona")
    }

    private fun wykonajAkcje() {
        val wejscie = etWejscie.text.toString().trim()

        if (wejscie.isEmpty()) {
            Toast.makeText(this, "Pole nie może być puste", Toast.LENGTH_SHORT).show()
            return
        }

        tvWynik.text = "Wynik: $wejscie"
    }
}

```

9. Obsługa zdarzeń

9.1 Toast — krótki komunikat

```

// Java
Toast.makeText(this, "Operacja zakończona!", Toast.LENGTH_SHORT).show();
Toast.makeText(this, "Wystąpił błąd", Toast.LENGTH_LONG).show();

// Z kontekstu klasy wewnętrznej:
Toast.makeText(MainActivity.this, "Komunikat", Toast.LENGTH_SHORT).show();

// Kotlin
Toast.makeText(this, "Operacja zakończona!", Toast.LENGTH_SHORT).show()

```

- `Toast.LENGTH_SHORT` — ok. 2 sekundy
- `Toast.LENGTH_LONG` — ok. 3,5 sekundy

9.2 TextWatcher — reagowanie na pisanie

Java:

```

getEmail().addTextChangedListener(new TextWatcher() {
    @Override
    public void beforeTextChanged(CharSequence s, int start, int count, int after) {}

    @Override
    public void onTextChanged(CharSequence s, int start, int before, int count) {
        // Wywołane przy każdym wciśnięciu klawisza
    }
})

```

```

        tvPodglad.setText("Wpisujesz: " + s.toString());
    }

    @Override
    public void afterTextChanged(Editable s) {
        // Po zakończeniu edycji znaków
        if (!s.toString().contains("@")) {
            etEmail.setError("Brak znaku @");
        }
    }
}
});

```

Kotlin:

```

etEmail.addTextChangedListener(object : TextWatcher {
    override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
    override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {
        tvPodglad.text = "Wpisujesz: $s"
    }
    override fun afterTextChanged(s: Editable?) {}
})

```

9.3 Kliknięcie na ImageView

Java:

```

ImageView iv = findViewById(R.id.ivKosc1);
iv.setOnClickListener(v → {
    // Kliknięto na obraz
    if (kosc.isDostepna()) {
        kosc.zablokuj();
        iv.setAlpha(0.5f);
    } else {
        kosc.odblokuj();
        iv.setAlpha(1.0f);
    }
});

```

Kotlin:

```

val iv = findViewById<ImageView>(R.id.ivKosc1)
iv.setOnClickListener {
    if (kosc.isDostepna()) {
        kosc.zablokuj()
        iv.alpha = 0.5f
    } else {
        kosc.odblokuj()
        iv.alpha = 1.0f
    }
}
}

```


9.4 Obsługa tablicy elementów w pętli

Gdy masz wiele podobnych elementów (np. 5 kości), używaj tablicy:

Java:

```
// Tablica ImageView
ImageView[] ivKosci = new ImageView[5];
ivKosci[0] = findViewById(R.id.ivKosc1);
ivKosci[1] = findViewById(R.id.ivKosc2);
ivKosci[2] = findViewById(R.id.ivKosc3);
ivKosci[3] = findViewById(R.id.ivKosc4);
ivKosci[4] = findViewById(R.id.ivKosc5);

// Przypisanie zdarzeń w pętli
for (int i = 0; i < 5; i++) {
    final int indeks = i; // WAŻNE: final dla lambda
    ivKosci[i].setOnClickListener(v → {
        // Kliknięto kość numer 'indeks'
        przełączKosc(indeks);
    });
}
```

Kotlin:

```
val ivKosci = arrayOf<ImageView>(
    findViewById(R.id.ivKosc1),
    findViewById(R.id.ivKosc2),
    findViewById(R.id.ivKosc3),
    findViewById(R.id.ivKosc4),
    findViewById(R.id.ivKosc5)
)

ivKosci.forEachIndexed { indeks, iv →
    iv.setOnClickListener {
        przełączKosc(indeks)
    }
}
```

9.5 Ustawianie koloru tła programowo

Java:

```
// Z kodu hex
view.setBackgroundColor(Color.parseColor("#D2691E"));

// Z RGB
view.setBackgroundColor(Color.rgb(210, 105, 30));

// Z zasobów (colors.xml)
view.setBackgroundColor(ContextCompat.getColor(this, R.color.colorPrimary));
```

Kotlin:

```
view.setBackgroundColor(Color.parseColor("#D2691E"))
view.setBackgroundColor(ContextCompat.getColor(this, R.color.colorPrimary))
```

10. Praca z listami

10.1 ArrayList — dynamiczna lista

Java:

```
ArrayList<String> lista = new ArrayList<>();

lista.add("Element 1");           // Dodaj na koniec
lista.add(0, "Element na początku"); // Dodaj na pozycję
lista.remove(0);                  // Usuń po indeksie
lista.remove("Element 1");        // Usuń po wartości

String el = lista.get(0);         // Pobierz element
int rozmiar = lista.size();       // Liczba elementów

// Sprawdzenie czy zawiera
if (lista.contains("szukane")) { ... }

// Iteracja
for (String s : lista) {
    System.out.println(s);
}

// Czyszczenie
lista.clear();
```

Kotlin:

```

val lista = ArrayList<String>()

lista.add("Element 1")
lista.add(0, "Na początku")
lista.removeAt(0)
lista.remove("Element 1")

val el = lista[0]
val rozmiar = lista.size

if ("szukane" in lista) { ... }

for (s in lista) println(s)
lista.forEach { println(it) }
lista.clear()

```

10.2 Wczytywanie z pliku assets

Umieść plik `dane.txt` w folderze `app/src/main/assets/`.

Java:

```

private ArrayList<String> wczytajZPliku(String nazwaPliku) {
    ArrayList<String> wynik = new ArrayList<>();
    try {
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(getAssets().open(nazwaPliku))
        );
        String linia;
        while ((linia = reader.readLine()) != null) {
            if (!linia.trim().isEmpty()) {
                wynik.add(linia.trim());
            }
        }
        reader.close();
    } catch (IOException e) {
        e.printStackTrace();
        Toast.makeText(this, "Błąd odczytu pliku", Toast.LENGTH_SHORT).show();
    }
    return wynik;
}

// Użycie:
ArrayList<String> notatki = wczytajZPliku("dane.txt");

```

Kotlin:

```

private fun wczytajZPliku(nazwaPliku: String): ArrayList<String> {
    val wynik = ArrayList<String>()
    try {
        val reader = assets.open(nazwaPliku).bufferedReader()
    }
}

```

```

        reader.forEachLine { linia →
            if (linia.trim().isEmpty()) wynik.add(linia.trim())
        }
        reader.close()
    } catch (e: IOException) {
        e.printStackTrace()
        Toast.makeText(this, "Błąd odczytu pliku", Toast.LENGTH_SHORT).show()
    }
    return wynik
}

```

11. Obrazy i zasoby graficzne

11.1 Umieszczanie obrazów w projekcie

- Wklej pliki `.png`, `.jpg` do folderu `res/drawable/`
- Nazwy plików: **tylko małe litery, cyfry, podkreślenie** (np. `kosc1.png`)
- Odwołanie w XML: `android:src="@drawable/kosc1"`

11.2 Dynamiczne ustawianie obrazu

Java — przez znane ID:

```
imageView.setImageResource(R.drawable.kosc3);
```

Java — przez dynamiczną nazwę:

```

// Np. "kosc" + 3 = "kosc3" → R.drawable.kosc3
int liczbaOczek = 3;
String nazwaZasobu = "kosc" + liczbaOczek;

int resId = getResources().getIdentifier(
    nazwaZasobu,    // np. "kosc3"
    "drawable",    // typ zasobu
    getPackageName() // pakiet aplikacji
);

if (resId != 0) {
    imageView.setImageResource(resId);
} else {
    Log.w("TAG", "Nie znaleziono zasobu: " + nazwaZasobu);
}

```

Kotlin:

```

val nazwaZasobu = "kosc$liczbaOczek"
val resId = resources.getIdentifier(nazwaZasobu, "drawable", packageName)
if (resId != 0) {

```

```
imageView.setImageResource(resId)
}
```

11.3 Przezroczystość (alpha)

```
// Java
imageView.setAlpha(0.0f); // całkowicie przezroczysty
imageView.setAlpha(0.5f); // 50% przezroczysty
imageView.setAlpha(1.0f); // w pełni widoczny

// Kotlin
imageView.alpha = 0.5f
```

11.4 Format kolorów hex z alpha

Format `#AARRGGBB` — 8 cyfr hex:

- `AA` — alpha (przezroczystość): `00` =pełna, `FF` =brak
- `RR` — czerwony
- `GG` — zielony
- `BB` — niebieski

Przykłady:

```
#ED27C121 → alpha=0xED=237/255=93%, kolor zielony #27C121
#80FF0000 → alpha=50%, kolor czerwony
#FFFFFFF → biały nieprzezroczysty
#00000000 → całkowicie przezroczysty
```

12. Kolory, style i motywy

12.1 Popularne kolory z zadań

```
←!— res/values/colors.xml →
<?xml version="1.0" encoding="utf-8"?>
<resources>

    ←!— Paleta: Gra w kości →
    <color name="tloKosci">#ED27C121</color>
    <color name="tloRzut">#ED275021</color>

    ←!— Paleta: Ciepłe kolory →
    <color name="beżowy">#F5F5DC</color>    ←!— Beige →
    <color name="brązowy">#A52A2A</color>    ←!— Brown →
    <color name="czekoladowy">#D2691E</color> ←!— Chocolate →

    ←!— Paleta: Chłodne odcienie (Smart Home) →
    <color name="jasnoniebieski">#ADD8E6</color> ←!— LightBlue →
    <color name="niebieskieNiebo">#87CEEB</color> ←!— SkyBlue →
    <color name="krolewski">#4169E1</color>    ←!— RoyalBlue →
    <color name="granatowy">#000080</color>    ←!— Navy →
```

```
    <!-- Paleta: Akcenty -->
    <color name="karmazyn">#DC143C</color>    <!-- Crimson -->

    <!-- Paleta: Ciemna zielen -->
    <color name="ciemnozielony">#558B2F</color>

</resources>
```

12.2 Ustawianie koloru w kodzie

Java:

```
// Kolor z hex
view.setBackgroundColor(Color.parseColor("#F5F5DC"));

// Kolor z RGB
view.setBackgroundColor(Color.rgb(245, 245, 220));

// Kolor z ARGB (z przezroczystością)
view.setBackgroundColor(Color.argb(237, 39, 193, 33)); // #ED27C121

// Kolor z zasobów colors.xml
view.setBackgroundColor(ContextCompat.getColor(this, R.color.bezowy));

// Kolor tekstu
textView.setTextColor(Color.parseColor("#A52A2A"));
textView.setTextColor(Color.WHITE);
textView.setTextColor(Color.RED);
```

Kotlin:

```
view.setBackgroundColor(Color.parseColor("#F5F5DC"))
view.setBackgroundColor(Color.rgb(245, 245, 220))
view.setBackgroundColor(ContextCompat.getColor(this, R.color.bezowy))
textView.setTextColor(Color.parseColor("#A52A2A"))
```

13. Walidacja danych

13.1 Sprawdzenie pustego pola

Java:

```
String tekst = editText.getText().toString().trim();

if (tekst.isEmpty()) {
    editText.setError("Pole nie może być puste");
    return; // Zatrzymaj dalsze wykonanie
}
```

Kotlin:

```
val tekst = editText.text.toString().trim()
if (tekst.isEmpty()) {
    editText.error = "Pole nie może być puste"
    return
}
```

13.2 Walidacja e-mail (sprawdzenie @)

Java:

```
String email = etEmail.getText().toString().trim();

if (!email.contains("@")) {
    tvStatus.setText("Błędny format e-mail – brak znaku @");
    tvStatus.setTextColor(Color.RED);
    return;
}
```

Kotlin:

```
val email = etEmail.text.toString().trim()
if (!email.contains("@")) {
    tvStatus.text = "Błędny format e-mail – brak znaku @"
    tvStatus.setTextColor(Color.RED)
    return
}
```

13.3 Walidacja haseł

Java:

```
String haslo1 = etHaslo.getText().toString();
String haslo2 = etPowtorzHaslo.getText().toString();

if (haslo1.isEmpty()) {
    tvStatus.setText("Podaj hasło");
    tvStatus.setTextColor(Color.RED);
    return;
}

if (!haslo1.equals(haslo2)) {
    tvStatus.setText("Hasła nie są identyczne");
    tvStatus.setTextColor(Color.RED);
    return;
}

// Sukces
```

```
tvStatus.setText("Rejestracja zakończona pomyślnie!");  
tvStatus.setTextColor(Color.parseColor("#4CAF50"));
```

Kotlin:

```
val haslo1 = etHaslo.text.toString()  
val haslo2 = etPowtorzHaslo.text.toString()  
  
if (haslo1.isEmpty()) {  
    tvStatus.text = "Podaj hasło"; tvStatus.setTextColor(Color.RED); return  
}  
if (haslo1 != haslo2) {  
    tvStatus.text = "Hasła nie są identyczne"; tvStatus.setTextColor(Color.RED); return  
}  
tvStatus.text = "Rejestracja zakończona pomyślnie!"  
tvStatus.setTextColor(Color.parseColor("#4CAF50"))
```

13.4 Konwersja tekstu na liczbę z obsługą błędów

Java:

```
String tekst = etLiczba.getText().toString().trim();  
  
if (tekst.isEmpty()) {  
    tvBlad.setText("Wpisz liczbę");  
    return;  
}  
  
int liczba;  
try {  
    liczba = Integer.parseInt(tekst);  
} catch (NumberFormatException e) {  
    tvBlad.setText("Nieprawidłowa liczba całkowita");  
    return;  
}  
  
if (liczba < 1 || liczba > 12) {  
    tvBlad.setText("Liczba musi być z zakresu 1-12");  
    return;  
}  
  
tvWynik.setText("Wpisano: " + liczba);
```

Kotlin:

```
val tekst = etLiczba.text.toString().trim()  
if (tekst.isEmpty()) { tvBlad.text = "Wpisz liczbę"; return }  
  
val liczba = tekst.toIntOrNull()  
if (liczba == null) { tvBlad.text = "Nieprawidłowa liczba"; return }  
if (liczba < 1 || liczba > 12) { tvBlad.text = "Zakres: 1-12"; return }
```



```
tvWynik.text = "Wpisano: $liczba"
```

13.5 Sprawdzanie numeru PESEL

Java:

```
public static boolean sprawdzSumeKontrolna(String pesel) {
    if (pesel == null || pesel.length() != 11) return false;

    int[] wagi = {1, 3, 7, 9, 1, 3, 7, 9, 1, 3};
    int suma = 0;

    for (int i = 0; i < 10; i++) {
        if (!Character.isDigit(pesel.charAt(i))) return false;
        suma += (pesel.charAt(i) - '0') * wagi[i];
    }

    int cyfraKontrolna = (10 - suma % 10) % 10;
    return cyfraKontrolna == (pesel.charAt(10) - '0');
}

public static char sprawdzPlec(String pesel) {
    // Cyfra na pozycji 9: parzysta = kobieta, nieparzysta = mężczyzna
    int cyfraPci = pesel.charAt(9) - '0';
    return (cyfraPci % 2 == 0) ? 'K' : 'M';
}
```

Kotlin:

```
fun sprawdzSumeKontrolna(pesel: String): Boolean {
    if (pesel.length != 11) return false
    val wagi = intArrayOf(1, 3, 7, 9, 1, 3, 7, 9, 1, 3)
    var suma = 0
    for (i in 0..9) {
        if (!pesel[i].isDigit()) return false
        suma += (pesel[i] - '0') * wagi[i]
    }
    val cyfraKontrolna = (10 - suma % 10) % 10
    return cyfraKontrolna == (pesel[10] - '0')
}

fun sprawdzPlec(pesel: String): Char {
    return if ((pesel[9] - '0') % 2 == 0) 'K' else 'M'
}
```

14. Klasy pomocnicze i logika biznesowa

14.1 Tworzenie klasy poza aktywnością

Dobłą praktyką jest oddzielenie logiki biznesowej od kodu interfejsu użytkownika (UI). Wynika to z wzorców projektowych takich jak MVC czy MVVM, gdzie widok (Activity) powinien zajmować się jedynie wyświetlaniem danych i reagowaniem na kliknięcia, a cała "matematyka" i przechowywanie stanu powinny znajdować się w osobnych klasach. Dzięki temu kod jest czytelniejszy, łatwiejszy do testowania i wielokrotnego użycia.

Dobłą praktyką jest oddzielenie logiki od kodu UI. Klasy logiki umieszczaj jako osobne pliki Java/Kotlin.

Java — klasa `Kosc.java`:

```
package com.example.mojaplikacja;

import java.util.Random;

/*
*****
* klasa:   Kosc
* opis:    Reprezentuje pojedynczą kość sześcienną.
*          Przechowuje wartość oczek i stan dostępności.
* pola:    liczbInstancji - statyczne, zlicza obiekty klasy
*          nazwyPlikow   - tablica nazw plików graficznych
*          liczbaOczek   - aktualna wartość kości (1-6 lub 0)
*          idPliku       - indeks grafiki w tablicy nazwyPlikow
*          dostepna      - czy kość może być rzucona
* autor:    12345
*****
*/
public class Kosc {

    // Pole statyczne – wspólne dla wszystkich instancji
    public static int liczbInstancji = 0;

    // Tablica nazw plików graficznych
    public String[] nazwyPlikow = {
        "kosc0", "kosc1", "kosc2",
        "kosc3", "kosc4", "kosc5", "kosc6"
    };

    // Pola prywatne – dostępne tylko wewnątrz klasy
    private int liczbaOczek;
    private int idPliku;
    private boolean dostepna;

    /*
    * Konstruktor jednoargumentowy.
    * Gdy argument poza zakresem 1-6, ustawia wartość 0.
    */
    public Kosc(int wartosc) {
        if (wartosc >= 1 && wartosc <= 6) {
            this.liczbaOczek = wartosc;
            this.idPliku = wartosc;
        } else {

```

```

        this.liczbaOczek = 0;
        this.idPliku = 0;
    }
    this.dostepna = true;
    liczbInstancji++;
}

/*
 * Konstruktor bezargumentowy – losuje wartość 1-6.
 */
public Kosc() {
    Random rand = new Random();
    int wylosowana = rand.nextInt(6) + 1;
    this.liczbaOczek = wylosowana;
    this.idPliku = wylosowana;
    this.dostepna = true;
    liczbInstancji++;
}

/*
 * Rzuca kością – losuje nową wartość.
 * Działa tylko gdy kość jest dostępna.
 */
public void rzuc() {
    if (dostepna) {
        Random rand = new Random();
        int wylosowana = rand.nextInt(6) + 1;
        this.liczbaOczek = wylosowana;
        this.idPliku = wylosowana;
    }
}

// Blokuje kość (ustawia jako niedostępną)
public void zablokuj() {
    this.dostepna = false;
}

// Odblokowuje kość
public void odblokuj() {
    this.dostepna = true;
}

// Zwraca wartość słownie
public String getWartoscSlownie() {
    switch (liczbaOczek) {
        case 1: return "jeden";
        case 2: return "dwa";
        case 3: return "trzy";
        case 4: return "cztery";
        case 5: return "pięć";
        case 6: return "sześć";
        default: return "zero";
    }
}
}

```

```

// Gettery
public int getLiczbaOczek() { return liczbaOczek; }
public int getIdPliku() { return idPliku; }
public boolean isDostepna() { return dostepna; }
public String getNazwePliku() { return nazwyPlikow[idPliku]; }
}

```

Kotlin — klasa Kosc.kt:

```

package com.example.mojaplikacja

import kotlin.random.Random

class Kosc {

    companion object {
        var liczbInstancji = 0
        val nazwyPlikow = arrayOf(
            "kosc0", "kosc1", "kosc2", "kosc3", "kosc4", "kosc5", "kosc6"
        )
    }

    var liczbaOczek: Int = 0
        private set
    var idPliku: Int = 0
        private set
    var dostepna: Boolean = true
        private set

    // Konstruktor jednoargumentowy (Java: new Kosc(3) → Kotlin: Kosc(3))
    constructor(wartość: Int) {
        if (wartość in 1..6) {
            liczbaOczek = wartość; idPliku = wartość
        } else {
            liczbaOczek = 0; idPliku = 0
        }
        dostepna = true
        liczbInstancji++
    }

    // Konstruktor bezargumentowy
    constructor() {
        val wylosowana = Random.nextInt(1, 7)
        liczbaOczek = wylosowana; idPliku = wylosowana
        dostepna = true
        liczbInstancji++
    }

    fun rzuc() {
        if (dostepna) {
            val wylosowana = Random.nextInt(1, 7)
            liczbaOczek = wylosowana; idPliku = wylosowana
        }
    }
}

```

```

    }
}

fun zablokuj() { dostepna = false }
fun odblokuj() { dostepna = true }

fun getWartoscSloownie() = when (liczbaOczek) {
    1 → "jeden"; 2 → "dwa"; 3 → "trzy"
    4 → "cztery"; 5 → "pięć"; 6 → "sześć"
    else → "zero"
}

fun getNazwePliku() = nazwyPlikow[idPliku]
}

```

14.2 Klasa z metodami statycznymi (narzędziowa)

Klasy narzędziowe (często nazywane Utils lub Helpers) służą do grupowania funkcji, które nie potrzebują przechowywać własnego stanu (nie mają pól instancyjnych). Są to metody uniwersalne, działające wyłącznie na podstawie podanych parametrów wejściowych (np. przeliczanie jednostek, formatowanie daty, walidacja tekstu). Wykorzystanie metod statycznych zapobiega konieczności niepotrzebnego tworzenia obiektów za każdym razem, gdy chcemy użyć danej funkcji.

Java:

```

public class Narzedzia {

    private Narzedzia() {} // Prywatny konstruktor – nie tworzy się instancji

    // Zliczanie samogłosek (polskie + angielskie)
    public static int liczSamogloski(String tekst) {
        if (tekst == null || tekst.isEmpty()) return 0;
        String samogloski = "aeiouAEIOUąęóĄąĘŁÓ";
        int licznik = 0;
        for (char c : tekst.toCharArray()) {
            if (samogloski.indexOf(c) ≥ 0) licznik++;
        }
        return licznik;
    }

    // Usuwanie sąsiednich powtórzeń
    public static String usunPowtorzenia(String tekst) {
        if (tekst == null || tekst.isEmpty()) return tekst;
        StringBuilder sb = new StringBuilder();
        sb.append(tekst.charAt(0));
        for (int i = 1; i < tekst.length(); i++) {
            if (tekst.charAt(i) ≠ tekst.charAt(i - 1)) {
                sb.append(tekst.charAt(i));
            }
        }
        return sb.toString();
    }
}

// Użycie (bez tworzenia obiektu):

```

```
int ile = Narzedzia.liczSamogloski("Ala ma kota"); // 5
String bez = Narzedzia.usunPowtorzenia("aabbcc"); // "abc"
```

Kotlin:

```
object Narzedzia {

    fun liczSamogloski(tekst: String?): Int {
        if (tekst.isNullOrEmpty()) return 0
        val samogloski = "aeiouAEIOUąęóĄĄĘĘŃ"
        return tekst.count { it in samogloski }
    }

    fun usunPowtorzenia(tekst: String?): String {
        if (tekst.isNullOrEmpty()) return tekst ?: ""
        val sb = StringBuilder()
        sb.append(tekst[0])
        for (i in 1 until tekst.length) {
            if (tekst[i] != tekst[i - 1]) sb.append(tekst[i])
        }
        return sb.toString()
    }
}

// Użycie:
val ile = Narzedzia.liczSamogloski("Ala ma kota")
val bez = Narzedzia.usunPowtorzenia("aabbcc")
```

14.3 Używanie klasy logiki w Activity

Java — integracja Kosc z MainActivity:

```
public class MainActivity extends AppCompatActivity {

    // Tablica obiektów logiki
    private Kosc[] kosci = new Kosc[5];
    private ImageView[] ivKosci = new ImageView[5];
    private TextView tvWynik;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Inicjalizacja obiektów logiki
        for (int i = 0; i < 5; i++) {
            kosci[i] = new Kosc(); // losuje wartość w konstruktorze
        }

        // Inicjalizacja widoków
        ivKosci[0] = findViewById(R.id.ivKosc1);
        ivKosci[1] = findViewById(R.id.ivKosc2);
```

```

        ivKosci[2] = findViewById(R.id.ivKosc3);
        ivKosci[3] = findViewById(R.id.ivKosc4);
        ivKosci[4] = findViewById(R.id.ivKosc5);
        tvWynik = findViewById(R.id.tvWynik);

        // Przycisk RZUT
        Button btnRzut = findViewById(R.id.btnRzut);
        btnRzut.setOnClickListener(v → wykonajRzut());

        // Kliknięcie na każdą kość
        for (int i = 0; i < 5; i++) {
            final int idx = i;
            ivKosci[i].setOnClickListener(v → przełączKosc(idx));
        }
    }

    private void wykonajRzut() {
        int suma = 0;
        for (int i = 0; i < 5; i++) {
            kosci[i].rzuc(); // Logika – rzut kością

            // UI – aktualizacja obrazka
            int resId = getResources().getIdentifier(
                "kosc" + kosci[i].getLiczbaOczek(),
                "drawable", getPackageName()
            );
            ivKosci[i].setImageResource(resId);

            suma += kosci[i].getLiczbaOczek();
        }
        tvWynik.setText(String.valueOf(suma));
    }

    private void przełączKosc(int idx) {
        if (kosci[idx].isDostepna()) {
            kosci[idx].zablokuj();
            ivKosci[idx].setAlpha(0.5f);
        } else {
            kosci[idx].odblokuj();
            ivKosci[idx].setAlpha(1.0f);
        }
    }
}

```

Kotlin:

```

class MainActivity : AppCompatActivity() {

    private val kosci = Array(5) { Kosc() }
    private lateinit var ivKosci: Array<ImageView>
    private lateinit var tvWynik: TextView

    override fun onCreate(savedInstanceState: Bundle?) {

```

```

super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)

ivKosci = arrayOf(
    findViewById(R.id.ivKosc1), findViewById(R.id.ivKosc2),
    findViewById(R.id.ivKosc3), findViewById(R.id.ivKosc4),
    findViewById(R.id.ivKosc5)
)
tvWynik = findViewById(R.id.tvWynik)

findViewById<Button>(R.id.btnRzut).setOnClickListener { wykonajRzut() }

ivKosci.forEachIndexed { idx, iv →
    iv.setOnClickListener { przełączKosc(idx) }
}
}

private fun wykonajRzut() {
    var suma = 0
    kosci.forEachIndexed { i, kosc →
        kosc.rzuc()
        val resId = resources.getIdentifier(
            "kosc${kosc.getLiczbaOczek()}", "drawable", packageName
        )
        ivKosci[i].setImageResource(resId)
        suma += kosc.getLiczbaOczek()
    }
    tvWynik.text = suma.toString()
}

private fun przełączKosc(idx: Int) {
    if (kosci[idx].isDostępna()) {
        kosci[idx].zablokuj(); ivKosci[idx].alpha = 0.5f
    } else {
        kosci[idx].odblokuj(); ivKosci[idx].alpha = 1.0f
    }
}
}
}

```

15. Dziedziczenie w Androidzie

15.1 Klasa bazowa i klasy pochodne

Java — `Urządzenie.java`:

```

public class Urządzenie {

    // Metoda publiczna dostępna w klasach pochodnych
    public void wyświetlKomunikat(String komunikat) {
        System.out.println(komunikat);
        // W Androidzie: Log.d("Urządzenie", komunikat);
    }
}

```



```
}  
}
```

Java — Pralka.java:

```
public class Pralka extends Urzadzenie {  
  
    // private = NIEDOSTĘPNE nawet w klasach pochodnych Pralki  
    private int numerProgramu = 0;  
  
    public int ustawProgram(int numer) {  
        if (numer ≥ 1 && numer ≤ 12) {  
            numerProgramu = numer;  
        } else {  
            numerProgramu = 0;  
        }  
        return numerProgramu;  
    }  
  
    public int getNumerProgramu() {  
        return numerProgramu;  
    }  
}
```

Java — Odkurzacz.java:

```
public class Odkurzacz extends Urzadzenie {  
  
    private boolean stanWlaczzenia = false;  
  
    public void on() {  
        if (!stanWlaczzenia) {  
            stanWlaczzenia = true;  
            wyswietlKomunikat("Odkurzacz włączono"); // metoda z klasy bazowej  
        }  
        // Gdy już włączony – nic nie robi  
    }  
  
    public void off() {  
        if (stanWlaczzenia) {  
            stanWlaczzenia = false;  
            wyswietlKomunikat("Odkurzacz wyłączono");  
        }  
        // Gdy już wyłączony – nic nie robi  
    }  
  
    public boolean isWlaczony() {  
        return stanWlaczzenia;  
    }  
}
```

Kotlin — klasy z dziedziczeniem:

```
open class Urzadzenie {
    // open = może być nadpisane w klasach pochodnych
    open fun wyswietlKomunikat(komunikat: String) {
        println(komunikat)
    }
}

class Pralka : Urzadzenie() {
    private var numerProgramu: Int = 0

    fun ustawProgram(numer: Int): Int {
        numerProgramu = if (numer in 1..12) numer else 0
        return numerProgramu
    }

    fun getNumerProgramu() = numerProgramu
}

class Odkurzacz : Urzadzenie() {
    private var stanWlaczania: Boolean = false

    fun on() {
        if (!stanWlaczania) {
            stanWlaczania = true
            wyswietlKomunikat("Odkurzacz włączono")
        }
    }

    fun off() {
        if (stanWlaczania) {
            stanWlaczania = false
            wyswietlKomunikat("Odkurzacz wyłączono")
        }
    }

    fun isWlaczony() = stanWlaczania
}
```

15.2 Użycie klas z dziedziczeniem w Activity

Java — integracja Pralka + Odkurzacz z UI:

```
public class MainActivity extends AppCompatActivity {

    // Obiekty logiki
    private Pralka pralka = new Pralka();
    private Odkurzacz odkurzacz = new Odkurzacz();

    private EditText etNrPrania;
    private TextView tvNumerPrania, tvStanOdkurzacza;
    private Button btnZatwierdz, btnWlacz;
```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    etNrPrania      = findViewById(R.id.etNrPrania);
    tvNumerPrania   = findViewById(R.id.tvNumerPrania);
    tvStanOdkurzacza = findViewById(R.id.tvStanOdkurzacza);
    btnZatwierdz    = findViewById(R.id.btnZatwierdz);
    btnWlacz        = findViewById(R.id.btnWlacz);

    // Pralka – ustawienie numeru programu
    btnZatwierdz.setOnClickListener(v → {
        String tekst = etNrPrania.getText().toString().trim();
        if (tekst.isEmpty()) {
            tvNumerPrania.setText("Numer prania: nie podano");
            return;
        }
        try {
            int numer = Integer.parseInt(tekst);
            int wynik = pralka.ustawProgram(numer);
            if (wynik != 0) {
                tvNumerPrania.setText("Numer prania: " + wynik);
            } else {
                tvNumerPrania.setText("Numer prania: nie podano");
            }
        } catch (NumberFormatException e) {
            tvNumerPrania.setText("Numer prania: nie podano");
        }
    });

    // Odkurzacze – włącz/wyłącz
    btnWlacz.setOnClickListener(v → {
        if (!odkurzacze.isWlaczony()) {
            odkurzacze.on();
            btnWlacz.setText("Wyłącz");
            tvStanOdkurzacza.setText("Odkurzacze włączony");
        } else {
            odkurzacze.off();
            btnWlacz.setText("Włącz");
            tvStanOdkurzacza.setText("Odkurzacze wyłączony");
        }
    });
}
}

```

16. Testy jednostkowe

16.1 Lokalizacja testów

```
app/src/  
├─ main/          ← Kod produkcyjny  
├─ test/          ← Testy jednostkowe (bez emulatora, szybkie)  
│   └─ java/com/example/.../  
│       └─ KosciTest.java  
└─ androidTest/   ← Testy instrumentalne (wymagają emulatora)
```

16.2 Zależności testowe w build.gradle

```
dependencies {  
    // JUnit 4 – framework do testów jednostkowych  
    testImplementation 'junit:junit:4.13.2'  
}
```

16.3 Podstawowy test JUnit 4

Java — KosciTest.java:

```
package com.example.mojaplikacja;  
  
import org.junit.Before;  
import org.junit.Test;  
import static org.junit.Assert.*;  
  
public class KosciTest {  
  
    private Kosci kosc;  
  
    // Wykonywane PRZED każdym testem  
    @Before  
    public void setUp() {  
        Kosci.liczbaInstancji = 0; // Reset licznika  
        kosc = new Kosci();  
    }  
  
    @Test  
    public void testWartoscRzutuWZakresie() {  
        kosc.rzuc();  
        int wartosc = kosc.getLiczbaOczek();  
        assertTrue("Wartość powinna być ≥ 1", wartosc ≥ 1);  
        assertTrue("Wartość powinna być ≤ 6", wartosc ≤ 6);  
    }  
  
    @Test  
    public void testBrakZmianyGdyKosciNiedostepna() {  
        kosc.rzuc();  
        int przed = kosc.getLiczbaOczek();
```

```

        kosc.zablokuj(); // Zablokuj kość
        kosc.rzuc();      // Rzut – nie powinien zmienić wartości
        kosc.rzuc();      // Kolejny rzut – nadal bez zmiany

        assertEquals("Wartość nie powinna się zmienić po zablokowaniu",
            przed, kosc.getLiczbaOczek());
    }

    @Test
    public void testKonstruktorZPrawidlowaWartoscia() {
        Kosc k5 = new Kosc(5);
        assertEquals(5, k5.getLiczbaOczek());
        assertEquals(5, k5.getIdPliku());
        assertTrue(k5.isDostepna());
    }

    @Test
    public void testKonstruktorZNieprawidlowaWartoscia() {
        Kosc kZla = new Kosc(7); // 7 jest poza zakresem 1-6
        assertEquals(0, kZla.getLiczbaOczek());
    }

    @Test
    public void testLicznikInstancji() {
        int przed = Kosc.liczbInstancji;
        new Kosc();
        new Kosc(3);
        assertEquals(przed + 2, Kosc.liczbInstancji);
    }

    @Test
    public void testBlokowanieOdblokowywanie() {
        assertTrue(kosc.isDostepna()); // Nowa kość – dostępna

        kosc.zablokuj();
        assertFalse(kosc.isDostepna()); // Po zablokowaniu – niedostępna

        kosc.odblokuj();
        assertTrue(kosc.isDostepna()); // Po odblokowaniu – dostępna
    }

    @Test
    public void testWartoscSloownie() {
        Kosc k3 = new Kosc(3);
        assertEquals("trzy", k3.getWartoscSloownie());

        Kosc k1 = new Kosc(1);
        assertEquals("jeden", k1.getWartoscSloownie());
    }
}

```

```
package com.example.mojaplikacja

import org.junit.Before
import org.junit.Test
import org.junit.Assert.*

class KoscTest {

    private lateinit var kosc: Kosc

    @Before
    fun setUp() {
        Kosc.liczbInstancji = 0
        kosc = Kosc()
    }

    @Test
    fun testWartoscRzutuWZakresie() {
        kosc.rzuc()
        val wartosc = kosc.liczbaOczek
        assertTrue("Wartość  $\geq$  1", wartosc  $\geq$  1)
        assertTrue("Wartość  $\leq$  6", wartosc  $\leq$  6)
    }

    @Test
    fun testBrakZmianyGdyKoscNiedostepna() {
        kosc.rzuc()
        val przed = kosc.liczbaOczek
        kosc.zablokuj()
        kosc.rzuc()
        kosc.rzuc()
        assertEquals("Wartość nie powinna się zmienić", przed, kosc.liczbaOczek)
    }

    @Test
    fun testKonstruktorZPrawidlowaWartoscia() {
        val k5 = Kosc(5)
        assertEquals(5, k5.liczbaOczek)
        assertTrue(k5.dostepna)
    }

    @Test
    fun testKonstruktorZNieprawidlowaWartoscia() {
        val kZla = Kosc(7)
        assertEquals(0, kZla.liczbaOczek)
    }
}
```

16.4 Adnotacje JUnit

ADNOTACJA	OPIS
<code>@Test</code>	Oznacza metodę testową — obowiązkowe
<code>@Before</code>	Wykonywana przed każdym testem (inicjalizacja)
<code>@After</code>	Wykonywana po każdym teście (sprzątanie)
<code>@BeforeClass</code>	Raz przed wszystkimi testami (metoda statyczna)
<code>@AfterClass</code>	Raz po wszystkich testach (metoda statyczna)
<code>@Ignore("powód")</code>	Pomija test

16.5 Asercje JUnit

```
// Równość
assertEquals(oczekiwana, faktyczna);
assertEquals("komunikat błędu", oczekiwana, faktyczna);
assertEquals(3.14, faktyczna, 0.001); // Dla double: z marginesem

// Prawda / Fałsz
assertTrue(warunek);
assertFalse(warunek);

// Null
assertNull(obiekt);
assertNotNull(obiekt);

// Tablice
assertArrayEquals(tablicaOczekiwana, tablicaFaktyczna);
```

16.6 Uruchamianie testów

1. Prawym przyciskiem na klasę testową → **Run 'NazwaTestu'**
2. Lub `Ctrl+Shift+F10` (Windows)
3. Wyniki: panel **Run** — zielony = OK, czerwony = błąd

17. Algorytmy — gotowe implementacje

Algorytmy to fundamentalna część programowania pozwalająca na skuteczne operowanie danymi. Znajomość podstawowych algorytmów ułatwia rozwiązywanie problemów analitycznych w aplikacjach, takich jak przetwarzanie list, wyszukiwanie informacji, czy kryptografia.

17.1 Sortowanie przez wybieranie (Selection Sort) — malejąco

Jak to działa i po co używać: Sortowanie przez wybieranie polega na iteracyjnym wyszukiwaniu największego (lub najmniejszego) elementu w nieposortowanej części tablicy i zamianie go z pierwszym elementem z tej części. Jest to algorytm mało wydajny dla bardzo wielkich zbiorów danych (złożoność $O(n^2)$), jednak jego prostota implementacji i brak konieczności używania dodatkowej pamięci sprawiają, że jest idealny do sortowania krótkich list lub w celach edukacyjnych.

Java:

```
public static void selectionSortMalejaco(int[] tab) {
    int n = tab.length;
    for (int i = 0; i < n - 1; i++) {
```

```

// Znajdź indeks maksimum w podtablicy od i do końca
int maxIdx = i;
for (int j = i + 1; j < n; j++) {
    if (tab[j] > tab[maxIdx]) {
        maxIdx = j;
    }
}
// Zamień maksimum na pozycję i
int tmp = tab[maxIdx];
tab[maxIdx] = tab[i];
tab[i] = tmp;
}
}

```

Kotlin:

```

fun selectionSortMalejaco(tab: IntArray) {
    val n = tab.size
    for (i in 0 until n - 1) {
        var maxIdx = i
        for (j in i + 1 until n) {
            if (tab[j] > tab[maxIdx]) maxIdx = j
        }
        val tmp = tab[maxIdx]; tab[maxIdx] = tab[i]; tab[i] = tmp
    }
}

```

17.2 Sortowanie bąbelkowe (Bubble Sort) — rosnąco

Jak to działa i po co używać: Algorytm wielokrotnie przechodzi przez listę, porównując sąsiadujące elementy i zamieniając je miejscami, jeśli są w niewłaściwej kolejności. Elementy o największej wartości "wypływają" na koniec listy jak bąbelki powietrza. Używany jest głównie ze względu na niezwykle intuicyjną logikę działania.

Java:

```

public static void bubbleSortRosnaco(int[] tab) {
    int n = tab.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (tab[j] > tab[j + 1]) {
                int tmp = tab[j];
                tab[j] = tab[j + 1];
                tab[j + 1] = tmp;
            }
        }
    }
}

```

Kotlin:


```

fun bubbleSortRosnaco(tab: IntArray) {
    val n = tab.size
    for (i in 0 until n - 1) {
        for (j in 0 until n - i - 1) {
            if (tab[j] > tab[j + 1]) {
                val tmp = tab[j]; tab[j] = tab[j + 1]; tab[j + 1] = tmp
            }
        }
    }
}

```

17.3 Wyszukiwanie liniowe z wartownikiem

Jak to działa i po co używać: Standardowe wyszukiwanie iteruje po tablicy sprawdzając za każdym razem dwa warunki: czy nie wyszliśmy poza tablicę oraz czy znaleźliśmy element. Dodanie "wartownika" na końcu tablicy (poszukiwanego elementu) eliminuje potrzebę sprawdzania czy wyszliśmy poza tablicę, co optymalizuje czas działania pętli. Służy do wydajnego znajdowania elementów w nieposortowanych zbiorach.

Java:

```

public static int szukajZWartownikiem(int[] tab, int szukana) {
    // Utwórz kopię z dodatkowym miejscem na wartownika
    int[] kopia = new int[tab.length + 1];
    System.arraycopy(tab, 0, kopia, 0, tab.length);
    kopia[tab.length] = szukana; // Wartownik – gwarancja zatrzymania pętli

    int i = 0;
    while (kopia[i] != szukana) {
        i++;
    }

    // Jeśli zatrzymaliśmy się przed wartownikiem – znaleziono
    return (i < tab.length) ? i : -1;
}

```

Kotlin:

```

fun szukajZWartownikiem(tab: IntArray, szukana: Int): Int {
    val kopia = IntArray(tab.size + 1)
    tab.copyInto(kopia)
    kopia[tab.size] = szukana // Wartownik

    var i = 0
    while (kopia[i] != szukana) i++
    return if (i < tab.size) i else -1
}

```

17.4 Algorytm Euklidesa — NWD

Jak to działa i po co używać: Największy Wspólny Dzielnik (NWD) używany jest do upraszczania ułamków, rozwiązywania problemów kryptograficznych oraz synchronizacji procesów. Algorytm Euklidesa to niesamowicie szybka metoda (korzystająca z reszty z dzielenia - modulo), która w kilku krokach pozwala znaleźć NWD.

Java:

```
public static int nwd(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

// Wersja rekurencyjna
public static int nwdRekurencja(int a, int b) {
    return b == 0 ? a : nwdRekurencja(b, a % b);
}
```

Kotlin:

```
fun nwd(a: Int, b: Int): Int {
    var x = a; var y = b
    while (y != 0) {
        val r = x % y; x = y; y = r
    }
    return x
}

fun nwdRekurencja(a: Int, b: Int): Int = if (b == 0) a else nwdRekurencja(b, a % b)
```

17.5 Sito Eratostenesa

Jak to działa i po co używać: To najpopularniejszy algorytm do znajdowania wszystkich liczb pierwszych w zadanym przedziale. Zamiast dzielić każdą liczbę, wykreślamy jej wielokrotności. Jest wykorzystywany w zabezpieczeniach (kryptografia RSA wymaga wielkich liczb pierwszych) oraz przy zaawansowanej optymalizacji operacji numerycznych.

Java:

```
public static boolean[] sitoEratostenesa(int n) {
    boolean[] czyPierwsza = new boolean[n + 1];

    // Inicjalizacja: zakładamy że wszystkie liczby ≥ 2 są pierwsze
    for (int i = 2; i ≤ n; i++) {
        czyPierwsza[i] = true;
    }

    // Wykreślanie wielokrotności
    for (int i = 2; (long)i * i ≤ n; i++) {
        if (czyPierwsza[i]) {
            for (int j = i * i; j ≤ n; j += i) {
                czyPierwsza[j] = false;
            }
        }
    }
}
```

```

        return czyPierwsza;
    }

    // Wypisanie liczb pierwszych
    public static void wypiszPierwsze(int n) {
        boolean[] sito = sitoEratostenesa(n);
        StringBuilder sb = new StringBuilder("Liczby pierwsze: ");
        for (int i = 2; i ≤ n; i++) {
            if (sito[i]) sb.append(i).append(" ");
        }
        System.out.println(sb.toString());
    }
}

```

Kotlin:

```

fun sitoEratostenesa(n: Int): BooleanArray {
    val czyPierwsza = BooleanArray(n + 1) { it ≥ 2 }
    var i = 2
    while (i.toLong() * i ≤ n) {
        if (czyPierwsza[i]) {
            var j = i * i
            while (j ≤ n) { czyPierwsza[j] = false; j += i }
        }
        i++
    }
    return czyPierwsza
}

```

17.6 Szyfr Cezara

Jak to działa i po co używać: Szyfr przesuwający polegający na zamianie każdej litery tekstu na literę oddaloną od niej o stałą wartość (klucz) w alfabecie. Obecnie nie ma wartości w zaawansowanym bezpieczeństwie, ale świetnie obrazuje podstawy kryptografii, manipulację znakami ASCII i operacje z dzieleniem modulo.

Java:

```

/**
 * Szyfruje tekst algorytmem Cezara.
 * Działa na małych literach a-z. Inne znaki (spacja, cyfry) – bez zmian.
 * Klucz może być ujemny lub większy niż 26.
 */
public static String szyfrCezara(String tekst, int klucz) {
    StringBuilder wynik = new StringBuilder();
    for (char c : tekst.toCharArray()) {
        if (c ≥ 'a' && c ≤ 'z') {
            // (c - 'a') – pozycja w alfabecie (0-25)
            // +klucz % 26 – przesunięcie z zawijaniem
            // +26 % 26 – obsługa ujemnych kluczy
            int nowaLitera = ((c - 'a' + klucz) % 26 + 26) % 26;
            wynik.append((char) ('a' + nowaLitera));
        } else {
            wynik.append(c); // Inne znaki bez zmian
        }
    }
}

```

```

    }

    }

    return wynik.toString();
}

// Deszyfrowanie: użyj klucza ujemnego
public static String deszyfrCezara(String szyfrogramm, int klucz) {
    return szyfrCezara(szyfrogramm, -klucz);
}

```

Kotlin:

```

fun szyfrCezara(tekst: String, klucz: Int): String {
    return tekst.map { c →
        if (c in 'a'..'z') {
            'a' + ((c - 'a' + klucz).mod(26))
        } else c
    }.joinToString("")
}

fun deszyfrCezara(tekst: String, klucz: Int) = szyfrCezara(tekst, -klucz)

```

17.7 Losowanie bez powtórzeń

Java:

```

public static int[] losujBezPowtorzen(int n, int max) {
    Random rand = new Random();
    int[] wynik = new int[n];
    boolean[] uzyte = new boolean[max + 1];
    int i = 0;

    while (i < n) {
        int wylosowana = rand.nextInt(max) + 1; // zakres 1..max
        if (!uzyte[wylosowana]) {
            wynik[i++] = wylosowana;
            uzyte[wylosowana] = true;
        }
    }

    return wynik;
}

// Np. losowanie 6 z 49:
// int[] zestaw = losujBezPowtorzen(6, 49);

```

Kotlin:

```

fun losujBezPowtorzen(n: Int, max: Int): IntArray {
    val wynik = IntArray(n)
    val uzyte = BooleanArray(max + 1)
    var i = 0

```

```
while (i < n) {  
    val los = (1..max).random()  
    if (!uzyte[los]) { wynik[i++] = los; uzyte[los] = true }  
}  
  
return wynik  
}
```

18. Typowe wzorce aplikacji

18.1 Formularz rejestracji z walidacją e-mail i hasła

Weryfikacja danych wprowadzanych przez użytkownika (walidacja) jest krytycznym elementem każdej aplikacji posiadającej formularze. Zabezpiecza ona aplikację przed błędami działania, atakami (np. typu wstrzykiwanie danych) oraz poprawia UX (User Experience) natychmiastowo informując o nieprawidłowo wypełnionych polach, zamiast pozwalać na wysłanie złych danych do serwera.

activity_main.xml (układ):

```
<?xml version="1.0" encoding="utf-8"?>  
<LinearLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:orientation="vertical"  
    android:padding="24dp">  
  
    <TextView  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:text="Rejestracja"  
        android:textSize="28sp"  
        android:textStyle="bold"  
        android:gravity="center"  
        android:layout_marginBottom="24dp" />  
  
    <EditText  
        android:id="@+id/etEmail"  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:hint="Adres e-mail"  
        android:inputType="textEmailAddress"  
        android:layout_marginBottom="12dp" />  
  
    <EditText  
        android:id="@+id/etHaslo"  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:hint="Hasło"  
        android:inputType="textPassword"  
        android:layout_marginBottom="12dp" />  
  
    <EditText  
        android:id="@+id/etPowtorzHaslo"
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Powtórz hasło"
        android:inputType="textPassword"
        android:layout_marginBottom="20dp" />

<Button
    android:id="@+id/btnZatwierdz"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="ZATWIERDŹ"
    android:layout_marginBottom="12dp" />

<TextView
    android:id="@+id/tvStatus"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center"
    android:textSize="16sp" />

</LinearLayout>

```

Java — logika walidacji:

```

public class MainActivity extends AppCompatActivity {

    private EditText etEmail, etHaslo, etPowtorzHaslo;
    private TextView tvStatus;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        etEmail      = findViewById(R.id.etEmail);
        etHaslo       = findViewById(R.id.etHaslo);
        etPowtorzHaslo = findViewById(R.id.etPowtorzHaslo);
        tvStatus      = findViewById(R.id.tvStatus);

        findViewById(R.id.btnZatwierdz).setOnClickListener(v → zatwierdz());
    }

    private void zatwierdz() {
        String email  = etEmail.getText().toString().trim();
        String haslo  = etHaslo.getText().toString();
        String powtorz = etPowtorzHaslo.getText().toString();

        if (!email.contains("@")) {
            pokazBlad("Błędny e-mail – brak znaku @");
            return;
        }
        if (haslo.isEmpty()) {
            pokazBlad("Podaj hasło");
        }
    }
}

```

```

        return;
    }

    if (!haslo.equals(powtorz)) {
        pokazBład("Hasła nie są identyczne");
        return;
    }

    pokazSukces("Rejestracja zakończona pomyślnie!");
}

private void pokazBład(String msg) {
    tvStatus.setText(msg);
    tvStatus.setTextColor(Color.RED);
}

private void pokazSukces(String msg) {
    tvStatus.setText(msg);
    tvStatus.setTextColor(Color.parseColor("#4CAF50"));
}
}

```

18.2 Galeria ze zdjęciami — nawigacja poprzedni/następny

Java:

```

public class GaleriaActivity extends AppCompatActivity {

    // Tablice zasobów i opisów
    private int[] obrazy = {
        R.drawable.oferta1, R.drawable.oferta2,
        R.drawable.oferta3, R.drawable.oferta4
    };

    private String[] opisy = {
        "Paryż – Wieża Eiffla",
        "Rzym – Koloseum",
        "Barcelona – Sagrada Família",
        "Amsterdam – Kanały"
    };

    private int aktualnyIndeks = 0;
    private ImageView ivObraz;
    private TextView tvOpis;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_galeria);

        ivObraz = findViewById(R.id.ivObraz);
        tvOpis = findViewById(R.id.tvOpis);

        aktualizujWidok(); // Wyświetl pierwszy obraz
    }
}

```

```

        findViewById(R.id.btnPoprzedni).setOnClickListener(v → {
            aktualnyIndeks = (aktualnyIndeks - 1 + obrazy.length) % obrazy.length;
            aktualizujWidok();
        });

        findViewById(R.id.btnNastepny).setOnClickListener(v → {
            aktualnyIndeks = (aktualnyIndeks + 1) % obrazy.length;
            aktualizujWidok();
        });
    }

    private void aktualizujWidok() {
        ivObraz.setImageResource(obrazy[aktualnyIndeks]);
        tvOpis.setText(opisy[aktualnyIndeks]);
    }
}

```

18.3 Suwak zmieniający rozmiar czcionki + cykl tekstów

Java:

```

public class CzcionkaActivity extends AppCompatActivity {

    private String[] cytaty = {"Dzień dobry", "Good morning", "Buenos dias"};
    private int aktualnyIndeks = 0;
    private TextView tvCytat;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_czcionka);

        tvCytat = findViewById(R.id.tvCytat);
        TextView tvRozmiar = findViewById(R.id.tvRozmiar);
        SeekBar sbRozmiar = findViewById(R.id.sbRozmiar);

        // Inicjalizacja aktualną wartością suwaka
        int startowy = sbRozmiar.getProgress();
        tvRozmiar.setText("Rozmiar: " + startowy);
        tvCytat.setTextSize(startowy);

        sbRozmiar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
            @Override
            public void onProgressChanged(SeekBar sb, int progress, boolean fromUser) {
                tvRozmiar.setText("Rozmiar: " + progress);
                tvCytat.setTextSize(progress);
            }
            @Override public void onStartTrackingTouch(SeekBar sb) {}
            @Override public void onStopTrackingTouch(SeekBar sb) {}
        });

        // Przycisk >> – cyklicznie zmienia tekst
    }
}

```



```

        Button btnNastepny = findViewById(R.id.btnNastepny);
        btnNastepny.setOnClickListener(v → {
            aktualnyIndeks = (aktualnyIndeks + 1) % cytaty.length;
            tvCytat.setText(cytaty[aktualnyIndeks]);
        });
    }
}

```

18.4 Lista notatek z wczytywaniem z pliku

Java:

```

public class NotatkiActivity extends AppCompatActivity {

    private ArrayList<String> listaNotatek;
    private ArrayAdapter<String> adapter;
    private EditText etNowaNotatka;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_notatki);

        etNowaNotatka = findViewById(R.id.etNowaNotatka);
        ListView lv = findViewById(R.id.lvNotatki);
        Button btnDodaj = findViewById(R.id.btnDodaj);

        listaNotatek = wczytajZPliku("dane.txt");

        adapter = new ArrayAdapter<>(this,
            android.R.layout.simple_list_item_1, listaNotatek);
        lv.setAdapter(adapter);

        btnDodaj.setOnClickListener(v → {
            String nowa = etNowaNotatka.getText().toString().trim();
            if (nowa.isEmpty()) {
                Toast.makeText(this, "Wpisz notatkę", Toast.LENGTH_SHORT).show();
                return;
            }
            listaNotatek.add(nowa);
            adapter.notifyDataSetChanged();
            etNowaNotatka.setText("");
        });
    }

    private ArrayList<String> wczytajZPliku(String plik) {
        ArrayList<String> wynik = new ArrayList<>();
        try {
            BufferedReader r = new BufferedReader(
                new InputStreamReader(getAssets().open(plik)));
            String linia;
            while ((linia = r.readLine()) ≠ null) {

```

```

        if (!linia.trim().isEmpty()) wynik.add(linia.trim());
    }
    r.close();
} catch (IOException e) {
    e.printStackTrace();
}
return wynik;
}
}

```

18.5 RGB Color Picker z trzema suwakami

Java:

```

public class ColorPickerActivity extends AppCompatActivity {

    private int r = 255, g = 0, b = 0;
    private View viewKolor;
    private TextView tvHex, tvR, tvG, tvB;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_color_picker);

        viewKolor = findViewById(R.id.viewKolor);
        tvHex = findViewById(R.id.tvHex);
        tvR = findViewById(R.id.tvR);
        tvG = findViewById(R.id.tvG);
        tvB = findViewById(R.id.tvB);

        SeekBar sbR = findViewById(R.id.sbRed);
        SeekBar sbG = findViewById(R.id.sbGreen);
        SeekBar sbB = findViewById(R.id.sbBlue);

        // Jeden listener dla wszystkich suwaków
        SeekBar.OnSeekBarChangeListener listener = new SeekBar.OnSeekBarChangeListener() {
            @Override
            public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
                int id = seekBar.getId();
                if (id == R.id.sbRed) { r = progress; tvR.setText(String.valueOf(r)); }
                else if (id == R.id.sbGreen) { g = progress; tvG.setText(String.valueOf(g)); }
                else if (id == R.id.sbBlue) { b = progress; tvB.setText(String.valueOf(b)); }
                aktualizujKolor();
            }
            @Override public void onStartTrackingTouch(SeekBar sb) {}
            @Override public void onStopTrackingTouch(SeekBar sb) {}
        };

        sbR.setOnSeekBarChangeListener(listener);
        sbG.setOnSeekBarChangeListener(listener);
        sbB.setOnSeekBarChangeListener(listener);
    }
}

```

```

        aktualizujKolor(); // Ustaw kolor startowy
    }

    private void aktualizujKolor() {
        int kolor = Color.rgb(r, g, b);
        viewKolor.setBackgroundColor(kolor);
        tvHex.setText(String.format("#%02X%02X%02X", r, g, b));
    }
}

```

18.6 Wzorzec pełnej dokumentacji metody w komentarzu

Dobłą praktyką programistyczną jest opatrzenie każdej ważnej metody komentarzem dokumentacyjnym, który wyjaśnia jej cel, parametry i zwracaną wartość:

```

/*
*****
* nazwa:   wykonajRzut
* opis:    Rzuci wszystkimi dostępnymi kośćmi i aktualizuje
*          UI – obrazki kości i sumę oczek.
* parametry: brak
* zwracany typ i opis: brak (void)
* autor:   Jan Kowalski
*****
*/
private void wykonajRzut() {
    // ...
}

```

Dla klasy:

```

/*
*****
* klasa:   Kosc
* opis:    Reprezentuje kość sześcienną do gry.
*          Przechowuje wartość oczek i plik graficzny.
* pola:    liczbInstancji - statyczne, licznik instancji
*          liczbaOczek    - aktualna wartość kości (0-6)
*          idPliku        - indeks grafiki w tablicy
*          dostepna       - stan dostępności kości
* autor:   12345
*****
*/

```

19. Uruchamianie i emulacja

19.1 Tworzenie emulatora Android (AVD)

1. Otwórz **Tools** → **Device Manager**
2. Kliknij **+** (Create Virtual Device) lub **Create Device**
3. Wybierz model telefonu: **Pixel 5** → **Next**
4. Wybierz wersję systemu (np. **API 33, x86_64**) — pobierz jeśli potrzebne
5. Kliknij **Finish**
6. Uruchom emulator klikając ► przy nazwie urządzenia

19.2 Uruchamianie aplikacji na emulatorze

1. Wybierz emulator z listy urządzeń (pasek narzędzi, góra ekranu)
2. Kliknij zielony trójkąt ► **Run** lub naciśnij **Shift+F10**
3. Poczekaj na zainstalowanie aplikacji — może potrwać kilka minut przy pierwszym uruchomieniu

*Jeśli emulator jest wolny: W ustawieniach AVD włącz **Hardware acceleration** (HAXM lub Hyper-V).*

19.3 Robienie zrzutów ekranu z emulatora

- Kliknij ikonę **Camera** 📷 w bocznym pasku emulatora
- Lub: w panelu bocznym Android Studio → zakładka **Emulator** → ikona screenshota

19.4 Logcat — podgląd komunikatów

Logcat wyświetla logi aplikacji w czasie rzeczywistym.

Dodawanie logów w kodzie:

```
// Java
import android.util.Log;

Log.d("TAG", "Komunikat debugowania");
Log.i("TAG", "Informacja");
Log.w("TAG", "Ostrzeżenie");
Log.e("TAG", "Błąd: " + e.getMessage());

// Kotlin
Log.d("TAG", "Komunikat debugowania")
Log.e("TAG", "Błąd", exception)
```

Przykład z wartościami:

```
Log.d("Kosc", "Wyrzucono: " + kosc.getLiczbaOczek()); // Kosc: Wyrzucono: 4
Log.d("Lista", "Rozmiar listy: " + lista.size());      // Lista: Rozmiar listy: 3
```

Filtrowanie w Logcat: wpisz tag (np. **MainActivity**) w pasku wyszukiwania.

19.5 Kompilacja aplikacji (Build)

- **Ctrl+F9** — zbuduj projekt bez uruchamiania
- **Build** → **Rebuild Project** — pełna kompilacja od nowa
- **Build** → **Clean Project** — usuń skompilowane pliki (pomaga przy dziwnych błędach)

20. Najczęstsze błędy i rozwiązania

20.1 NullPointerException

Przyczyna: Wywołanie `findViewById()` przed `setContentView()` lub zły identyfikator.

```
// BŁĄD – brak setContentView przed findViewById
Button btn = findViewById(R.id.btnOK); // null!
setContentView(R.layout.activity_main);

// POPRAWNIE
setContentView(R.layout.activity_main);
Button btn = findViewById(R.id.btnOK); // OK
```

20.2 NumberFormatException

Przyczyna: Próba konwersji pustego lub nieprawidłowego tekstu na liczbę.

```
// BŁĄD
int n = Integer.parseInt(""); // NumberFormatException!

// POPRAWNIE
String tekst = editText.getText().toString().trim();
if (!tekst.isEmpty()) {
    try {
        int n = Integer.parseInt(tekst);
    } catch (NumberFormatException e) {
        Toast.makeText(this, "Nieprawidłowa liczba", Toast.LENGTH_SHORT).show();
    }
}
```

20.3 Aktywność nie uruchamia się

Przyczyna: Brak wpisu w `AndroidManifest.xml`.

←!— Sprawdź czy aktywność jest zadeklarowana w manifeście →

```
<activity android:name=".NazwaAktywnosci" />
```

20.4 Obraz nie wyświetla się

Przyczyna: Błędna nazwa pliku graficznego.

```
// Sprawdź – jeśli resId == 0, zasób nie istnieje
int resId = getResources().getIdentifier("kosc3", "drawable", getPackageName());
if (resId == 0) {
    Log.e("TAG", "Nie znaleziono zasobu 'kosc3'");
    return;
}
iv.setImageResource(resId);
```

Zasady nazewnictwa plików w drawable:

-  `kosc1.png`, `tlo_aplikacji.jpg`, `ikona123.png`
-  `Kosc1.png` (wielka litera), `kość.png` (ó), `mój-obraz.png` (myślnik)

20.5 Lista nie aktualizuje się po dodaniu elementu

Przyczyna: Brak wywołania `notifyDataSetChanged()`.

```
lista.add("Nowy element");
adapter.notifyDataSetChanged(); // OBOWIĄZKOWE!
```

20.6 Zmiana UI z wątku w tle

Przyczyna: Bezpośrednia zmiana widoków z nie-UI wątku.

```
// BŁĄD - crash w wątku roboczym
new Thread() → {
    tvWynik.setText("Gotowe!"); // CalledFromWrongThreadException!
}.start();

// POPRAWNIE
new Thread() → {
    runOnUiThread() → tvWynik.setText("Gotowe!");
}.start();
```

20.7 ScrollView nie przewija

Przyczyna: Bezpośredni element potomny ScrollView ma `android:layout_height="match_parent"`.

```
←!— BŁĄD —→
<ScrollView ...>
    <LinearLayout android:layout_height="match_parent"> ←!— zawsze match_parent! —→

←!— POPRAWNIE —→
<ScrollView ...>
    <LinearLayout android:layout_height="wrap_content"> ←!— dostosuj do treści —→
```

Ściągowka — kluczowe operacje w kodzie

Java vs Kotlin — porównanie składni

OPERACJA	JAVA	KOTLIN
Deklaracja zmiennej	<code>String s = "text";</code>	<code>val s = "text" lub var s = "text"</code>
Pobranie tekstu	<code>et.getText().toString().trim()</code>	<code>et.text.toString().trim()</code>
Ustawienie tekstu	<code>tv.setText("Tekst")</code>	<code>tv.text = "Tekst"</code>
Konwersja String→int	<code>Integer.parseInt(s)</code>	<code>s.toInt() lub s.toIntOrNull()</code>
Konwersja int→String	<code>String.valueOf(n)</code>	<code>n.toString() lub "\$n"</code>
Format hex	<code>String.format("#%02X", n)</code>	<code>"#%02X".format(n)</code>
Losowanie liczby	<code>new Random().nextInt(6) + 1</code>	<code>(1..6).random()</code>
Sprawdzenie pustego	<code>s.isEmpty()</code>	<code>s.isEmpty() lub s.isNullOrEmpty()</code>
Kliknięcie przycisku	<code>btn.setOnClickListener(v → {...})</code>	<code>btn.setOnClickListener { ... }</code>

Najważniejsze metody

```
// Pobieranie tekstu z EditText
String tekst = editText.getText().toString().trim();

// Ustawianie obrazu po dynamicznej nazwie ("kosc" + liczba)
int resId = getResources().getIdentifier("kosc" + n, "drawable", getPackageName());
imageView.setImageResource(resId);

// Przezroczystość ImageView
imageView.setAlpha(0.5f); // 50% przezroczysty
imageView.setAlpha(1.0f); // pełna widoczność

// Kolor z hex
view.setBackgroundColor(Color.parseColor("#D2691E"));

// Toast
Toast.makeText(this, "Komunikat", Toast.LENGTH_SHORT).show();

// Logowanie
Log.d("TAG", "Wartość: " + zmienna);

// Odczyt pliku z assets
BufferedReader r = new BufferedReader(
    new InputStreamReader(getAssets().open("dane.txt")));

// Losowanie w zakresie 1-6
int los = new Random().nextInt(6) + 1;

// Wyczyść pole EditText
editText.setText("");

// Ukryj / pokaż element
view.setVisibility(View.GONE);
```

```
view.setVisibility(View.VISIBLE);

// Zmiana tekstu przycisku
button.setText("Wyłącz");

// Pobierz zaznaczony RadioButton
int id = radioGroup.getCheckedRadioButtonId(); // -1 jeśli żaden
```

Checklist przed oddaniem aplikacji mobilnej

- ☐ Plik layout XML — poprawna struktura, wszystkie elementy mają unikalne `id`
- ☐ Kolory tła — zastosowano konsekwentną paletę barw (hex z `#`)
- ☐ Marginesy — ustawione logicznie, z zachowaniem czytelności i proporcji
- ☐ Rozmiary czcionek — w `sp`, rozmiary elementów w `dp`
- ☐ Wyśrodkowanie — `android:gravity="center"` tam gdzie wymagane
- ☐ Klasy logiki — zaimportowane i używane w Activity (nie powielać logiki w UI)
- ☐ Walidacja danych wejściowych — obsługa pustych pól, błędnych wartości
- ☐ Toast / komunikaty — informują użytkownika o błędach
- ☐ Komentarze dokumentacyjne — dodane nad kluczowymi metodami i klasami logicznymi w celu lepszej czytelności kodu
- ☐ Logika dziedziczenia — klasy pochodne wywołują metody bazowe przez `super`
- ☐ Zrzuty ekranu — wszystkie stany aplikacji, pełny ekran z paskiem zadań
- ☐ Archiwum projektu — spakowany jako `mobilna.zip`
- ☐ Plik `readme.txt` — podstawowa dokumentacja środowiska: OS, IDE, emulator, język programowania
- ☐ Kompilacja — aplikacja się kompiluje i uruchamia na emulatorze

Dokumentacja obejmuje tworzenie projektów, strukturę i zawartość plików, budowanie interfejsu użytkownika w XML, widżety, obsługę zdarzeń, pracę z listami i obrazami, klasy logiki z przykładami Java i Kotlin, dziedziczenie, testy jednostkowe oraz kompletne wzorce typowych aplikacji mobilnych.

21. Nawigacja między aktywnościami (Intent)

21.1 Przejście do innej aktywności

Intent (zamiar) to fundamentalny mechanizm systemu Android służący do komunikacji między komponentami. Za pomocą Intentów aplikacja "informuje" system operacyjny o tym, co chce zrobić (np. otworzyć nowy ekran, zrobić zdjęcie, otworzyć przeglądarkę).

Dlaczego Intent a nie zwykłe tworzenie obiektu (np. `new Aktywnosc()`)? Aktywności w Androidzie są zarządzane w 100% przez system operacyjny, ponieważ posiadają złożony cykl życia oraz powiązane zasoby pamięci. Nie możemy ich po prostu stworzyć w kodzie; musimy poprosić system (przez Intent), by zainicjalizował dany ekran za nas. — m.in. otwarcie nowej aktywności.

Java:

```
// Przejście z MainActivity do DrugaActivity
Intent intent = new Intent(this, DrugaActivity.class);
startActivity(intent);

// Zamknięcie bieżącej aktywności po przejściu
finish();
```


Kotlin:

```
val intent = Intent(this, DrugaActivity::class.java)
startActivity(intent)
finish()
```

21.2 Przekazywanie danych przez Intent (Extras)

Aktywność wysyłająca (Java):

```
Intent intent = new Intent(this, DrugaActivity.class);

// Dodaj dane do intentu jako "extras" – klucz i wartość
intent.putExtra("KLUCZ_IMIE", "Anna");
intent.putExtra("KLUCZ_WIEK", 30);
intent.putExtra("KLUCZ_AKTYWNY", true);
intent.putExtra("KLUCZ_WYNIK", 3.14);

startActivity(intent);
```

Aktywność odbierająca (Java):

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_druga);

    // Pobierz Intent, który uruchomił tę aktywność
    Intent intent = getIntent();

    // Odczytaj wartości (drugi argument to wartość domyślna)
    String imie    = intent.getStringExtra("KLUCZ_IMIE");        // null jeśli brak
    int wiek       = intent.getIntExtra("KLUCZ_WIEK", 0);        // 0 jeśli brak
    boolean aktyw  = intent.getBooleanExtra("KLUCZ_AKTYWNY", false);
    double wynik   = intent.getDoubleExtra("KLUCZ_WYNIK", 0.0);

    tvImie.setText("Witaj, " + imie + "! Masz " + wiek + " lat.");
}
```

Kotlin — wysyłanie:

```
val intent = Intent(this, DrugaActivity::class.java).apply {
    putExtra("KLUCZ_IMIE", "Anna")
    putExtra("KLUCZ_WIEK", 30)
    putExtra("KLUCZ_AKTYWNY", true)
}
startActivity(intent)
```

Kotlin — odbieranie:

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_druga)

    val imie    = intent.getStringExtra("KLUCZ_IMIE") ?: "Nieznany"
    val wiek    = intent.getIntExtra("KLUCZ_WIEK", 0)
    val aktywny = intent.getBooleanExtra("KLUCZ_AKTYWNY", false)

    tvImie.text = "Witaj, $imie! Masz $wiek lat."
}

```

21.3 Przekazywanie tablicy przez Intent

Java:

```

// Wysłanie tablicy
int[] wyniki = {1, 5, 3, 2, 6};
intent.putExtra("KLUCZ_WYNIKI", wyniki);

// Odbieranie tablicy
int[] wyniki = intent.getIntArrayExtra("KLUCZ_WYNIKI");

```

Kotlin:

```

// Wysłanie
intent.putExtra("KLUCZ_WYNIKI", intArrayOf(1, 5, 3, 2, 6))

// Odbieranie
val wyniki = intent.getIntArrayExtra("KLUCZ_WYNIKI")

```

21.4 Pełny przykład: przejście z wynikiem gry

MainActivity → WynikActivity:

```

// W MainActivity – po zakończeniu gry
private void pokazWynik(int suma, int liczbaRzutow) {
    Intent intent = new Intent(this, WynikActivity.class);
    intent.putExtra("SUMA", suma);
    intent.putExtra("LICZBA_RZUTOW", liczbaRzutow);
    intent.putExtra("SREDNIA", (double) suma / liczbaRzutow);
    startActivity(intent);
}

```

```

// W WynikActivity
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_wynik);
}

```

```

int suma      = getIntent().getIntExtra("SUMA", 0);
int rzuty     = getIntent().getIntExtra("LICZBA_RZUTOW", 1);
double srednia = getIntent().getDoubleExtra("SREDNIA", 0.0);

tvSuma.setText("Suma: " + suma);
tvRzuty.setText("Rzuty: " + rzuty);
tvSrednia.setText("Średnia: " + String.format("%.2f", srednia));

// Przycisk powrotu
btnPowrot.setOnClickListener(v → finish()); // zamknij WynikActivity, wróć do gry
}

```

22. Przechowywanie danych

22.1 SharedPreferences — zapis prostych wartości

SharedPreferences przechowuje pary klucz-wartość trwale między uruchomieniami aplikacji. Dane nie znikają po zamknięciu aplikacji.

Java:

```

// === ZAPIS ===
SharedPreferences prefs = getSharedPreferences("MojeUstawienia", MODE_PRIVATE);
SharedPreferences.Editor editor = prefs.edit();

editor.putString("IMIE",      "Jan Kowalski");
editor.putInt("POZIOM",      5);
editor.putFloat("OBJETOSC",   0.8f);
editor.putBoolean("MUZYKAWL", true);
editor.putLong("DATA",        System.currentTimeMillis());

editor.apply(); // Asynchroniczny zapis – preferowane
// lub editor.commit(); // Synchroniczny zapis – blokuje wątek

// === ODCZYT ===
SharedPreferences prefs = getSharedPreferences("MojeUstawienia", MODE_PRIVATE);

String imie      = prefs.getString("IMIE", "Gość"); // "Gość" = domyślna
int poziom       = prefs.getInt("POZIOM", 1);
float objetosc   = prefs.getFloat("OBJETOSC", 1.0f);
boolean muzyka   = prefs.getBoolean("MUZYKAWL", true);

// === USUWANIE ===
editor.remove("IMIE"); // Usuń jeden klucz
editor.apply();

editor.clear(); // Usuń wszystkie dane
editor.apply();

```

Kotlin:

```

val prefs = getSharedPreferences("MojeUstawienia", MODE_PRIVATE)

// Zapis
prefs.edit().apply {
    putString("IMIE", "Jan Kowalski")
    putInt("POZIOM", 5)
    putBoolean("MUZYKAWL", true)
    apply()
}

// Odczyt
val imie    = prefs.getString("IMIE", "Gość") ?: "Gość"
val poziom  = prefs.getInt("POZIOM", 1)
val muzyka  = prefs.getBoolean("MUZYKAWL", true)

```

22.2 Przykład: zapamiętywanie wyniku gry

```

// Zapis najlepszego wyniku po każdym rzucie
private void zapiszNajlepszyWynik(int nowyWynik) {
    SharedPreferences prefs = getSharedPreferences("GraPrefs", MODE_PRIVATE);
    int obecnyNajlepszy = prefs.getInt("NAJLEPSZY_WYNIK", 0);

    if (nowyWynik > obecnyNajlepszy) {
        prefs.edit().putInt("NAJLEPSZY_WYNIK", nowyWynik).apply();
        tvNajlepszy.setText("Nowy rekord: " + nowyWynik);
    }
}

// Wczytanie najlepszego wyniku przy starcie aplikacji
@Override
protected void onCreate(Bundle savedInstanceState) {
    // ...
    SharedPreferences prefs = getSharedPreferences("GraPrefs", MODE_PRIVATE);
    int najlepszy = prefs.getInt("NAJLEPSZY_WYNIK", 0);
    tvNajlepszy.setText("Rekord: " + najlepszy);
}

```

23. Pełne przykłady klas OOP

23.1 Klasa Notatka (Zarządzanie notatkami z auto-inkrementacją ID)

```

/*
*****
* klasa:   Notatka
* opis:    Reprezentuje notatkę tekstową z tytułem i treścią.
*          Automatycznie przydziela unikalne ID na podstawie licznika.
* poła:    licznikNotatek - statyczne, zlicza wszystkie notatki
*          id              - prywatne, unikalny identyfikator

```

```

*      tytuł    - chronione, tytuł notatki
*      treść    - chronione, treść notatki
* autor: 12345
*****
*/

public class Notatka {

    // Statyczne – wspólne dla wszystkich instancji, niedostępne dla klas pochodnych
    private static int licznikNotatek = 0;

    // Prywatne – dostępne tylko w tej klasie
    private int id;

    // Chronione – dostępne w tej klasie i klasach pochodnych
    protected String tytuł;
    protected String treść;

    /*
    * Konstruktor jednoargumentowy – tworzy notatkę z tytułem i treścią.
    * Inkrementuje licznik i ustawia ID.
    */
    public Notatka(String tytuł, String treść) {
        licznikNotatek++;
        this.id = licznikNotatek; // Pierwsza notatka ma id=1, druga id=2 itd.
        this.tytuł = tytuł;
        this.treść = treść;
    }

    /*
    * Wyświetla tytuł i treść notatki.
    */
    public void wyswietl() {
        System.out.println("Tytuł: " + tytuł);
        System.out.println("Treść: " + treść);
    }

    /*
    * Metoda diagnostyczna – wyświetla wszystkie pola oddzielone średnikami.
    */
    public void diagnostyka() {
        System.out.println(licznikNotatek + ";" + id + ";" + tytuł + ";" + treść);
    }

    // Gettery
    public static int getLicznikNotatek() { return licznikNotatek; }
    public int getId() { return id; }
    public String getTytuł() { return tytuł; }
    public String getTreść() { return treść; }
}

```

Kotlin:

```

class Notatka(val tytuł: String, val tresc: String) {

    companion object {
        var licznikNotatek = 0
    }

    val id: Int

    init {
        licznikNotatek++
        id = licznikNotatek
    }

    fun wyswietl() {
        println("Tytuł: $tytuł")
        println("Treść: $tresc")
    }

    fun diagnostyka() {
        println("$licznikNotatek;$id;$tytuł;$tresc")
    }
}

```

Program główny testujący:

```

public static void main(String[] args) {

    Notatka n1 = new Notatka("Urodziny", "Urodziny Ali w sobotę");
    Notatka n2 = new Notatka("Zakupy", "Mleko, chleb, masło");

    n1.wyswietl();
    n1.diagnostyka();
    n2.wyswietl();
    n2.diagnostyka();

    System.out.println("Łącznie notatek: " + Notatka.getLicznikNotatek());
}

```

Oczekiwane wyjście:

```

Tytuł: Urodziny
Treść: Urodziny Ali w sobotę
2;1;Urodziny;Urodziny Ali w sobotę
Tytuł: Zakupy
Treść: Mleko, chleb, masło
2;2;Zakupy;Mleko, chleb, masło
Łącznie notatek: 2

```

23.2 Hierarchia klas Urządzenie (Dziedziczenie i polimorfizm w systemie Smart Home)

Urządzenie.java:

```
public class Urządzenie {

    protected String nazwaUrządzenia;

    public Urządzenie(String nazwaUrządzenia) {
        this.nazwaUrządzenia = nazwaUrządzenia;
    }

    public void wyświetlKomunikat(String komunikat) {
        System.out.println("[ " + nazwaUrządzenia + " ] " + komunikat);
    }

    public String getNazwa() {
        return nazwaUrządzenia;
    }
}
```

Pralka.java:

```
public class Pralka extends Urządzenie {

    // private – klasy pochodne Pralki nie mają dostępu
    private int numerProgramu;
    private boolean włączona;

    public Pralka(String nazwa) {
        super(nazwa);
        this.numerProgramu = 0;
        this.włączona = false;
    }

    /**
     * Ustawia numer programu prania (zakres 1-12).
     * Poza zakresem – ustawia 0.
     * @return ustawiony numer programu lub 0
     */
    public int ustawProgram(int numer) {
        if (numer ≥ 1 && numer ≤ 12) {
            this.numerProgramu = numer;
        } else {
            this.numerProgramu = 0;
        }
        return this.numerProgramu;
    }

    public int getNumerProgramu() { return numerProgramu; }
    public boolean isWłączona() { return włączona; }
}
```

Odkurzacz.java:

```
public class Odkurzacz extends Urzadzenie {

    private boolean stanWlaczania;

    public Odkurzacz(String nazwa) {
        super(nazwa);
        this.stanWlaczania = false;
    }

    /*
     * Włącza odkurzacz jeśli był wyłączony.
     * Jeśli już włączony – nie robi nic.
     */
    public void on() {
        if (!stanWlaczania) {
            stanWlaczania = true;
            wyswietlKomunikat("Włączono"); // metoda z klasy bazowej
        }
    }

    /*
     * Wyłącza odkurzacz jeśli był włączony.
     */
    public void off() {
        if (stanWlaczania) {
            stanWlaczania = false;
            wyswietlKomunikat("Wyłączono");
        }
    }

    public boolean isWlaczony() { return stanWlaczania; }
}
```

Klimatyzator.java:

```
public class Klimatyzator extends Urzadzenie {

    private int temperatura; // zakres np. 16-30
    private boolean wlaczony;

    public Klimatyzator(String nazwa) {
        super(nazwa);
        this.temperatura = 22; // domyślna temperatura
        this.wlaczony = false;
    }

    public void on() {
        if (!wlaczony) {
            wlaczony = true;
            wyswietlKomunikat("Włączono. Temperatura: " + temperatura + "°C");
        }
    }
}
```



```

    }
}

public void off() {
    if (włączony) {
        włączony = false;
        wyswietlKomunikat("Wyłączono");
    }
}

/*
 * Ustawia temperaturę w zakresie 16-30.
 * @return nowa temperatura lub -1 jeśli poza zakresem
 */
public int ustawTemperature(int temp) {
    if (temp ≥ 16 && temp ≤ 30) {
        this.temperatura = temp;
        return temp;
    }
    return -1;
}

public int getTemperatura() { return temperatura; }
public boolean isWłączony() { return włączony; }
}

```

Użycie w MainActivity — Smart Home:

```

public class MainActivity extends AppCompatActivity {

    private Pralka pralka = new Pralka("Pralka Samsung");
    private Odkurzacz odkurzacz = new Odkurzacz("Roomba 960");
    private Klimatyzator klima = new Klimatyzator("Daikin FTX25");

    private TextView tvPralka, tvOdkurzacz, tvKlima;
    private EditText etNumerProgramu, etTemperatura;
    private Button btnPralkaZatwierdz, btnOdkurzaczToggle, btnKlimaZatwierdz;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        tvPralka = findViewById(R.id.tvPralka);
        tvOdkurzacz = findViewById(R.id.tvOdkurzacz);
        tvKlima = findViewById(R.id.tvKlima);
        etNumerProgramu = findViewById(R.id.etNumerProgramu);
        etTemperatura = findViewById(R.id.etTemperatura);
        btnPralkaZatwierdz = findViewById(R.id.btnPralkaZatwierdz);
        btnOdkurzaczToggle = findViewById(R.id.btnOdkurzaczToggle);
        btnKlimaZatwierdz = findViewById(R.id.btnKlimaZatwierdz);

        // Pralka – ustaw program

```

```

btnPralkaZatwierdz.setOnClickListener(v → {
    String tekst = etNumerProgramu.getText().toString().trim();
    if (tekst.isEmpty()) {
        tvPralka.setText("Numer prania: nie podano");
        return;
    }
    try {
        int numer = Integer.parseInt(tekst);
        int wynik = pralka.ustawProgram(numer);
        if (wynik ≠ 0) {
            tvPralka.setText("Numer prania: " + wynik);
        } else {
            tvPralka.setText("Numer prania: nie podano");
        }
    } catch (NumberFormatException e) {
        tvPralka.setText("Numer prania: nie podano");
    }
});

// Odkurzacze – włącz/wyłącz
btnOdkurzaczeToggle.setOnClickListener(v → {
    if (!odkurzacze.isWlaczony()) {
        odkurzacze.on();
        btnOdkurzaczeToggle.setText("Wyłącz odkurzacze");
        tvOdkurzacze.setText("Odkurzacze: włączony");
    } else {
        odkurzacze.off();
        btnOdkurzaczeToggle.setText("Włącz odkurzacze");
        tvOdkurzacze.setText("Odkurzacze: wyłączony");
    }
});

// Klimatyzator – ustaw temperaturę
btnKlimatyzatorZatwierdz.setOnClickListener(v → {
    String tekst = etTemperatura.getText().toString().trim();
    if (tekst.isEmpty()) { tvKlimatyzator.setText("Podaj temperaturę"); return; }
    try {
        int temp = Integer.parseInt(tekst);
        int wynik = klimatyzator.ustawTemperaturę(temp);
        if (wynik ≠ -1) {
            if (!klimatyzator.isWlaczony()) klimatyzator.on();
            tvKlimatyzator.setText("Klimatyzator: " + wynik + "°C");
        } else {
            tvKlimatyzator.setText("Temperatura poza zakresem (16-30)");
        }
    } catch (NumberFormatException e) {
        tvKlimatyzator.setText("Nieprawidłowa temperatura");
    }
});
});
}
}

```

23.3 Klasa BankKonto z walidacją

```
public class BankKonto {

    private static int licznikKont = 0;
    private int numer;
    private String wlasciciel;
    private double saldo;

    public BankKonto(String wlasciciel, double saloPoczkowe) {
        licznikKont++;
        this.numer = licznikKont;
        this.wlasciciel = wlasciciel;
        this.saldo = (salopoczkowe >= 0) ? salopoczkowe : 0;
    }

    public boolean wplac(double kwota) {
        if (kwota <= 0) return false;
        saldo += kwota;
        return true;
    }

    public boolean wyplac(double kwota) {
        if (kwota <= 0 || kwota > saldo) return false;
        saldo -= kwota;
        return true;
    }

    public boolean przelej(BankKonto cel, double kwota) {
        if (wyplac(kwota)) {
            cel.wplac(kwota);
            return true;
        }
        return false;
    }

    public static int getLicznikKont() { return licznikKont; }
    public int getNumer() { return numer; }
    public String getWlasciciel() { return wlasciciel; }
    public double getSaldo() { return saldo; }

    @Override
    public String toString() {
        return String.format("Konto #%d | %s | %.2f zł", numer, wlasciciel, saldo);
    }
}
```

Kotlin data class:

```
// Data class – automatycznie generuje: toString, equals, hashCode, copy
data class Produkt(
    val id: Int,
```

```

    val nazwa: String,
    val cena: Double,
    val kategoria: String
) {
    fun getOpisSlowny(): String {
        return "$nazwa ($kategoria) – ${"%0.2f".format(cena)} zł"
    }
}

// Użycie
val prod = Produkt(1, "Laptop", 3999.99, "Elektronika")
println(prod.getOpisSlowny())

// Kopia z jedną zmienioną właściwością
val tanszyProd = prod.copy(cena = 2999.99)
println(tanszyProd)

```

24. Kotlin — specyficzna składnia i możliwości

24.1 Val vs Var

```

val imie = "Jan"      // NIEZMIENNE (jak final w Java)
var wiek = 25         // ZMIENNE

imie = "Anna"         // Błąd kompilacji!
wiek = 26             // OK

```

24.2 Typy nullable i operator ?.

```

var tekst: String? = null // ? = może być null

// Bezpieczne wywołanie – nic nie robi jeśli null
val dlugosc = tekst?.length // null jeśli tekst == null, liczba jeśli nie

// Elvis operator – wartość domyślna gdy null
val d = tekst?.length ?: 0 // 0 gdy tekst == null

// Twarde rzutowanie (rzuca NullPointerException jeśli null)
val d2 = tekst!!.length

```

24.3 When — rozszerzone switch

```

val wartosc = 4

val wynik = when (wartosc) {
    1    → "jeden"
    2, 3 → "dwa lub trzy"
    in 4..6 → "cztery do sześciu"
}

```

```

        else → "inne"
    }

    // When bez argumentu – jak seria if-else
    val kategoria = when {
        wiek < 18 → "Małoletni"
        wiek < 65 → "Dorosły"
        else     → "Senior"
    }

```

24.4 For loops i range

```

// Pętla po zakresie
for (i in 0..9) print(i)          // 0,1,2,...,9
for (i in 0 until 9) print(i)    // 0,1,...,8 (bez 9)
for (i in 9 downTo 0) print(i)   // 9,8,...,0
for (i in 0..10 step 2) print(i) // 0,2,4,6,8,10

// Pętla po kolekcji
val lista = listOf("A", "B", "C")
for (el in lista) println(el)

lista.forEach { println(it) }
lista.forEachIndexed { i, el → println("$i: $el") }

// Mapa
val mapa = mapOf("a" to 1, "b" to 2)
for ((klucz, wartosc) in mapa) {
    println("$klucz = $wartosc")
}

```

24.5 Extension Functions — rozszerzenia klas

```

// Dodaj metodę do klasy String bez dziedziczenia
fun String.liczbaSamoglosek(): Int {
    val samogloski = "aeiouąęóAEIOUAÆĖĖŲ"
    return this.count { it in samogloski }
}

fun String.odwroc(): String = this.reversed()

fun String.byMoze(prefix: String = "Może: ") = "$prefix$this"

// Rozszerzenie EditText
fun android.widget.EditText.pobierzTekst() = this.text.toString().trim()
fun android.widget.EditText.wyczysc()      = this.setText("")
fun android.widget.EditText.ustawBład(msg: String) { this.error = msg }

// Rozszerzenie Int
fun Int.jestPierwsza(): Boolean {
    if (this < 2) return false
}

```

```

    for (i in 2..Math.sqrt(this.toDouble()).toInt()) {
        if (this % i == 0) return false
    }

    return true
}

// Użycie
"Ała ma kota".liczbaSamoglosek() // 5
"hello".odwroc()                  // "olleh"
17.jestPierwsza()                  // true
etImie.pobierzTekst()              // bez .getText().toString().trim()
etImie.wyczyszc()                  // bez .setText("")

```

24.6 Lambda i Higher-order functions

```

// Lambda jako zmienna
val podwoj: (Int) → Int = { x → x * 2 }
val suma: (Int, Int) → Int = { a, b → a + b }

// Wywołanie
podwoj(5)      // 10
suma(3, 4)     // 7

// Funkcja wyższego rzędu – przyjmuje lambdę jako parametr
fun wykonajDlaKazdegoKosci(kosci: Array<Kosc>, akcja: (Kosc, Int) → Unit) {
    kosci.forEachIndexed { idx, kosc → akcja(kosc, idx) }
}

// Wywołanie
wykonajDlaKazdegoKosci(kosci) { kosc, idx →
    kosc.rzuc()
    ivKosci[idx].setImageResource(getResId("kosc${kosc.liczbaOczek}"))
}

```

25. Kompletne layouty XML dla typowych zadań

25.1 Layout: Gra w kości (5 kości + rzut + wynik)

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#ED27C121">

    <!-- Tytuł gry -->
    <TextView

        android:id="@+id/tvTytuł"
        android:layout_width="match_parent"

```

```
android:layout_height="wrap_content"
android:text="Gra w kości"
android:textSize="28sp"
android:textColor="#FFFFFF"
android:textStyle="bold"
android:gravity="center"
android:layout_margin="10dp" />
```

←!— Rząd 5 kości —→

<LinearLayout

```
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="horizontal"
android:background="#FFFFFF"
android:padding="4dp">
```

<ImageView android:id="@+id/ivKosc1"

```
android:layout_width="0dp" android:layout_height="60dp"
android:layout_weight="1" android:layout_margin="9dp"
android:src="@drawable/kosc0" android:scaleType="fitCenter" />
```

<ImageView android:id="@+id/ivKosc2"

```
android:layout_width="0dp" android:layout_height="60dp"
android:layout_weight="1" android:layout_margin="9dp"
android:src="@drawable/kosc0" android:scaleType="fitCenter" />
```

<ImageView android:id="@+id/ivKosc3"

```
android:layout_width="0dp" android:layout_height="60dp"
android:layout_weight="1" android:layout_margin="9dp"
android:src="@drawable/kosc0" android:scaleType="fitCenter" />
```

<ImageView android:id="@+id/ivKosc4"

```
android:layout_width="0dp" android:layout_height="60dp"
android:layout_weight="1" android:layout_margin="9dp"
android:src="@drawable/kosc0" android:scaleType="fitCenter" />
```

<ImageView android:id="@+id/ivKosc5"

```
android:layout_width="0dp" android:layout_height="60dp"
android:layout_weight="1" android:layout_margin="9dp"
android:src="@drawable/kosc0" android:scaleType="fitCenter" />
```

</LinearLayout>

←!— Etykieta sumy —→

<TextView

```
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Suma oczek:"
android:textSize="18sp"
android:textColor="#FFFFFF"
android:gravity="center"
android:layout_marginTop="10dp" />
```

←!— Wynik —→

```
<TextView
    android:id="@+id/tvWynik"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="0"
    android:textSize="52sp"
    android:textColor="#FFFFFF"
    android:textStyle="bold"
    android:gravity="center"
    android:layout_margin="10dp" />
```

←!— Przycisk RZUT →

```
<Button
    android:id="@+id/btnRzut"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="RZUT"
    android:textSize="20sp"
    android:textColor="#FFFFFF"
    android:backgroundTint="#E0275021"
    android:layout_margin="10dp"
    android:padding="14dp" />
```

←!— Liczba rzutów →

```
<TextView
    android:id="@+id/tvLiczbaRzutow"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Rzuty: 0"
    android:textSize="14sp"
    android:textColor="#E0E0E0"
    android:gravity="center" />
```

```
</LinearLayout>
```

25.2 Layout: Smart Home (4 urządzenia)

```
<?xml version="1.0" encoding="utf-8"?>

<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#87CEEB">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <!-- Nagłówek -->
```



```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Smart Home"
    android:textSize="26sp"
    android:textStyle="bold"
    android:textColor="#000080"
    android:gravity="center"
    android:layout_marginBottom="20dp" />
```

<!-- PRALKA -->

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="🔵 Pralka"
    android:textSize="18sp"
    android:textStyle="bold"
    android:layout_marginBottom="4dp" />
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginBottom="4dp">
```

```
    <EditText
        android:id="@+id/etNumerProgramu"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:hint="Nr programu (1-12)"
        android:inputType="number"
        android:layout_marginEnd="8dp" />
```

```
    <Button
        android:id="@+id/btnPralkaZatwierdz"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="USTAW" />
```

```
</LinearLayout>
```

```
<TextView
    android:id="@+id/tvPralka"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Numer prania: nie podano"
    android:textSize="14sp"
    android:layout_marginBottom="16dp" />
```

<!-- ODKURZACZ -->

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
```

```

        android:text="🔴 0dkurzacz"
        android:textSize="18sp"
        android:textStyle="bold"
        android:layout_marginBottom="4dp" />

<Button
    android:id="@+id/btn0dkurzaczToggle"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Włącz odkurzacz"
    android:layout_marginBottom="4dp" />

<TextView
    android:id="@+id/tv0dkurzacz"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="0dkurzacz: wyłączony"
    android:textSize="14sp"
    android:layout_marginBottom="16dp" />

<!-- ≡ KLIMATYZATOR ≡ -->
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="🌀 Klimatyzator"
    android:textSize="18sp"
    android:textStyle="bold"
    android:layout_marginBottom="4dp" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginBottom="4dp">

    <EditText
        android:id="@+id/etTemperatura"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:hint="Temperatura (16-30)"
        android:inputType="number"
        android:layout_marginEnd="8dp" />

    <Button
        android:id="@+id/btnKlimaZatwierdz"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="USTAW" />

</LinearLayout>

<TextView
    android:id="@+id/tvKlima"

```

```
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Klimatyzator: wyłączony"
        android:textSize="14sp" />

    </LinearLayout>

</ScrollView>
```

25.3 Layout: Formularz rejestracji ze scrollem

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Rejestracja"
            android:textSize="28sp"
            android:textStyle="bold"
            android:textColor="#333333"
            android:gravity="center"
            android:layout_marginBottom="32dp" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="E-mail"
            android:textSize="14sp"
            android:textColor="#555555" />

        <EditText
            android:id="@+id/etEmail"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="adres@example.com"
            android:inputType="textEmailAddress"
            android:layout_marginBottom="16dp" />

        <TextView
```

```

        android:text="Hasło (min. 6 znaków)"
        android:textSize="14sp"
        android:textColor="#555555" />

<EditText
    android:id="@+id/etHaslo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="....."
    android:inputType="textPassword"
    android:layout_marginBottom="16dp" />

<EditText
    android:id="@+id/etPowtorzHaslo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Powtórz hasło"
    android:inputType="textPassword"
    android:layout_marginBottom="24dp" />

<Button
    android:id="@+id/btnZatwierdz"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="ZAREJESTRUJ"
    android:textSize="16sp"
    android:layout_marginBottom="12dp" />

<TextView
    android:id="@+id/tvStatus"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text=""
    android:textSize="14sp"
    android:gravity="center"
    android:padding="8dp" />

</LinearLayout>

</ScrollView>

```

25.4 Layout: Galeria z nawigacją

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#1A1A2E"
    android:padding="16dp">

```

←!— Tytuł —>

```
<TextView
    android:id="@+id/tvTytułGalerii"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Galeria"
    android:textSize="24sp"
    android:textStyle="bold"
    android:textColor="#E0E0E0"
    android:gravity="center"
    android:layout_marginBottom="16dp" />
```

←!— Obraz główny —>

```
<ImageView
    android:id="@+id/ivGłówny"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:scaleType="fitCenter"
    android:layout_marginBottom="12dp" />
```

←!— Opis —>

```
<TextView
    android:id="@+id/tvOpis"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text=""
    android:textSize="16sp"
    android:textColor="#E0E0E0"
    android:gravity="center"
    android:layout_marginBottom="12dp" />
```

←!— Licznik: 1 z 4 —>

```
<TextView
    android:id="@+id/tvLicznik"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="1 / 4"
    android:textSize="14sp"
    android:textColor="#AAAAAA"
    android:gravity="center"
    android:layout_marginBottom="12dp" />
```

←!— Przyciski nawigacji —>

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <Button
        android:id="@+id/btnPoprzedni"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
```

```

        android:layout_weight="1"
        android:text="← POPRZEDNI"
        android:layout_marginEnd="8dp" />

        <Button
            android:id="@+id/btnNastepny"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="NASTĘPNY →" />

    </LinearLayout>

</LinearLayout>

```

Java — implementacja galerii:

```

public class GaleriaActivity extends AppCompatActivity {

    private int[] obrazy = {
        R.drawable.oferta1, R.drawable.oferta2,
        R.drawable.oferta3, R.drawable.oferta4
    };

    private String[] opisy = {
        "Paryż – Wieża Eiffla",
        "Rzym – Koloseum",
        "Barcelona – Sagrada Família",
        "Amsterdam – Kanały"
    };

    private int aktualny = 0;

    private ImageView ivGlowny;
    private TextView tvOpis, tvLicznik;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_galeria);

        ivGlowny = findViewById(R.id.ivGlowny);
        tvOpis = findViewById(R.id.tvOpis);
        tvLicznik = findViewById(R.id.tvLicznik);

        aktualizujWidok(); // Wyświetl pierwszy

        findViewById(R.id.btnPoprzedni).setOnClickListener(v → {
            aktualny = (aktualny - 1 + obrazy.length) % obrazy.length;
            aktualizujWidok();
        });

        findViewById(R.id.btnNastepny).setOnClickListener(v → {
            aktualny = (aktualny + 1) % obrazy.length;
            aktualizujWidok();
        });
    }
}

```

```

});

}

private void aktualizujWidok() {
    ivGlowny.setImageResource(obrazy[aktualny]);
    tvOpis.setText(opisy[aktualny]);
    tvLicznik.setText((aktualny + 1) + " / " + obrazy.length);
}
}

```

26. Konwersje i operacje na danych — gotowe fragmenty

26.1 Konwersje typów

```

// ≡ Java ≡

// String → int (z obsługą błędów)
int n = Integer.parseInt("42");
int n2 = Integer.parseInt(tekst, 10); // base 10

// String → double
double d = Double.parseDouble("3.14");

// int → String
String s = String.valueOf(42);
String s2 = Integer.toString(42);
String s3 = "" + 42; // konkatencja z pustym stringiem

// double → String (z formatowaniem)
String s4 = String.format("%.2f", 3.14159); // "3.14"
String s5 = String.format("%.0f", 3.7); // "4"

// char → int (wartość ASCII)
int ascii = (int) 'A'; // 65

// int → char
char c = (char) 65; // 'A'

// boolean → String
String b = String.valueOf(true); // "true"

```

```

// ≡ Kotlin ≡

// String → Int (null jeśli niepowodzenie)
val n = "42".toIntOrNull() // Int? 42 lub null
val n2 = "xyz".toIntOrNull() // null
val n3 = "42".toInt() // Int 42, wyjątek jeśli błąd

// String → Double

```

```

val d = "3.14".toDoubleOrNull()    // Double?
val d2 = "3.14".toDouble()

// Liczba → String
val s = 42.toString()
val s2 = "%.2f".format(3.14159)    // "3.14"
val s3 = "$liczba zł"              // interpolacja

// char → Int
val ascii = 'A'.code                // 65

// Int → Char
val c = 65.toChar()                 // 'A'

```

26.2 Operacje na Stringach

```

// Java
String tekst = " Witaj, Świecie! ";

tekst.trim()                // usuwa spacje z początku i końca
tekst.toLowerCase()         // zamiana na małe litery
tekst.toUpperCase()         // zamiana na wielkie litery
tekst.length()              // długość
tekst.charAt(0)              // pierwszy znak
tekst.substring(2, 7)        // wycinek: indeksy 2-6
tekst.contains("Świecie")    // czy zawiera
tekst.startsWith(" Witaj")  // czy zaczyna się od
tekst.endsWith("! ")        // czy kończy się na
tekst.replace("Świecie", "Java") // zamiana
tekst.split(", ")            // podział na tablicę
tekst.isEmpty()              // czy pusta
tekst.equals("inny tekst")   // porównanie (nie ==!)
tekst.equalsIgnoreCase("WITAJ, ŚWIECIE!") // bez uwzględnienia wielkości
tekst.indexOf("Świecie")     // indeks pierwszego wystąpienia

// Składanie Stringa – StringBuilder (szybsze niż +)
StringBuilder sb = new StringBuilder();
sb.append("Witaj, ");
sb.append("Świecie");
sb.append("!");
String wynik = sb.toString(); // "Witaj, Świecie!"

```

```

// Kotlin
val tekst = " Witaj, Świecie! "

tekst.trim()
tekst.lowercase()
tekst.uppercase()
tekst.length
tekst[0]                // operator indeksowania
tekst.substring(2, 7)

```



```

tekst.contains("Świecie")
tekst.replace("Świecie", "Kotlin")
tekst.split(", ")
tekst.isEmpty()
tekst.isNotEmpty()
tekst.isBlank()           // tylko spacje lub pusta
tekst.isNotBlank()
tekst = "inny tekst"      // == w Kotlin porównuje wartość (jak equals w Java)
tekst.equals("WITAJ", ignoreCase = true)

// Interpolacja stringów (Kotlin)
val imie = "Jan"; val wiek = 30
println("Witaj, $imie! Masz $wiek lat.")
println("Za 10 lat będziesz miał ${wiek + 10} lat.")

```

26.3 Operacje na tablicach

```

// Java
int[] tab = {5, 2, 8, 1, 9, 3};

tab.length           // liczba elementów
tab[0]               // pierwszy element
tab[tab.length-1]    // ostatni element

// Iteracja
for (int x : tab) System.out.print(x + " ");

// Kopiowanie
int[] kopia = Arrays.copyOf(tab, tab.length);
int[] kopiaZakresowa = Arrays.copyOfRange(tab, 1, 4); // indeksy 1,2,3

// Sortowanie (rosnąco)
Arrays.sort(tab);

// Wypełnienie
Arrays.fill(tab, 0); // wypełnij zerami

// Wyszukiwanie (tylko w posortowanej!)
int idx = Arrays.binarySearch(tab, 5);

// Konwersja do Stringa
System.out.println(Arrays.toString(tab)); // [1, 2, 3, 5, 8, 9]

```

```

// Kotlin
val tab = intArrayOf(5, 2, 8, 1, 9, 3)

tab.size           // liczba elementów
tab[0]             // pierwszy
tab.last()         // ostatni
tab.first()        // pierwszy

```

```

tab.sorted()                // zwraca posortowaną listę (bez modyfikacji)
tab.sortedDescending()      // malejąco
tab.sum()                   // suma
tab.max()                   // maksimum
tab.min()                   // minimum
tab.average()               // średnia
tab.count { it > 3 }        // zlicz spełniające warunek
tab.filter { it > 3 }       // filtruj

tab.contentToString()       // "[5, 2, 8, 1, 9, 3]"
tab.copyOf()                // pełna kopia
tab.copyOfRange(1, 4)       // kopia zakresu

```

26.4 Generowanie liczb losowych

```

// Java
import java.util.Random;

Random rand = new Random();

int los1 = rand.nextInt(6) + 1;    // 1-6 (kość sześcienna)
int los2 = rand.nextInt(100);      // 0-99
int los3 = rand.nextInt(51) + 50;  // 50-100
double losD = rand.nextDouble();   // 0.0-1.0

// Losowanie elementu z tablicy
String[] kolory = {"czerwony", "niebieski", "zielony"};
String wylosowany = kolory[rand.nextInt(kolory.length)];

// Przetasowanie tablicy (Fisher-Yates)
for (int i = tab.length - 1; i > 0; i--) {
    int j = rand.nextInt(i + 1);
    int tmp = tab[i]; tab[i] = tab[j]; tab[j] = tmp;
}

```

```

// Kotlin
import kotlin.random.Random

val los1 = (1..6).random()          // 1-6
val los2 = (0 until 100).random()    // 0-99
val los3 = (50..100).random()        // 50-100
val losD = Random.nextDouble()       // 0.0-1.0

val kolory = arrayOf("czerwony", "niebieski", "zielony")
val wylosowany = kolory.random()     // losowy element tablicy

val lista = mutableListOf(1, 2, 3, 4, 5)
lista.shuffle()                      // przetasowanie

```

27. Wzorce importów — co importować

27.1 Standardowe importy Java w projekcie Android

```
// Aktywność
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

// Widżety – podstawowe
import android.widget.TextView;
import android.widget.EditText;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.ListView;
import android.widget.ArrayAdapter;
import android.widget.Spinner;
import android.widget.AdapterView;
import android.widget.SeekBar;
import android.widget.CheckBox;
import android.widget.RadioButton;
import android.widget.RadioGroup;
import android.widget.Toast;
import android.widget.ProgressBar;

// Widżety zaawansowane
import androidx.recyclerview.widget.RecyclerView;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.appcompat.app.AlertDialog;
import android.view.LayoutInflater;
import android.view.ViewGroup;
import android.view.View;

// Kolory i grafika
import android.graphics.Color;
import androidx.core.content.ContextCompat;

// Tekst
import android.text.TextWatcher;
import android.text.Editable;
import android.text.InputType;

// Intent i nawigacja
import android.content.Intent;
import android.content.SharedPreferences;

// Zasoby
import android.content.res.Resources;

// Kolekcje
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Random;
import java.util.Collections;
```

```
// IO (pliki)
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.io.FileOutputStream;
import java.io.FileInputStream;

// Logowanie
import android.util.Log;
```

27.2 Importy Kotlin

```
// Aktywność
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

// Widżety
import android.widget.*
import androidx.recyclerview.widget.RecyclerView
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.appcompat.app.AlertDialog

// Kolory
import android.graphics.Color
import androidx.core.content.ContextCompat

// Kolekcje i losowanie
import kotlin.random.Random

// IO
import java.io.BufferedReader
import java.io.InputStreamReader
import java.io.IOException

// Logowanie
import android.util.Log
```

28. Skróty klawiszowe Android Studio

AKCJA	WINDOWS/LINUX
Uruchom aplikację	Shift+F10
Zbuduj projekt	Ctrl+F9
Automatyczne formatowanie kodu	Ctrl+Alt+L
Dodaj brakujące importy	Alt+Enter
Autouzupełnianie	Ctrl+Space
Komentarz linii	Ctrl+/
Zaznacz linię	Ctrl+W
Duplikuj linię	Ctrl+D
Usuń linię	Ctrl+Y
Szukaj w plikach	Ctrl+Shift+F
Idź do definicji	Ctrl+B lub Ctrl+Click
Refaktoryzacja (zmień nazwę)	Shift+F6
Generuj kod (getter, setter)	Alt+Insert
Zwiń/rozwiń blok kodu	Ctrl+- / Ctrl++
Przenieś linię w górę/dół	Shift+Alt+↑/↓
Znajdź i zastąp	Ctrl+H
Uruchom wszystkie testy	Ctrl+Shift+F10
Pokaż błędy projektu	Ctrl+Alt+Shift+I

29. Typowe struktury pełnych projektów

29.1 Projekt: Gra w kości — pełna lista plików

```
app/src/main/
├─ java/com/example/grawkosci/
│   ├── MainActivity.java    ← Aktywność (UI + obsługa zdarzeń)
│   └─ Kosc.java            ← Klasa logiki kości
├─ res/
│   ├── drawable/
│   │   ├── kosc0.png        ← Kość "zero" (szara/pusta)
│   │   ├── kosc1.png
│   │   ├── kosc2.png
│   │   ├── kosc3.png
│   │   ├── kosc4.png
│   │   ├── kosc5.png
│   │   └─ kosc6.png
│   ├── layout/
│   │   └─ activity_main.xml  ← Layout ekranu gry
│   └─ values/
```

```
|   |   | colors.xml
|   |   | strings.xml
|   | manifests/
|   |   | AndroidManifest.xml
```

29.2 Projekt: Smart Home — pełna lista plików

```
app/src/main/
|   | java/com/example/smarthome/
|   |   | MainActivity.java      ← Aktywność główna
|   |   | Urzadzenie.java       ← Klasa bazowa
|   |   | Pralka.java           ← Klasa pochodna
|   |   | Odkurzacz.java        ← Klasa pochodna
|   |   | Klimatyzator.java     ← Klasa pochodna (opcjonalnie)
|   | res/
|   |   | drawable/
|   |   |   | pralka.png
|   |   |   | odkurzacz.png
|   |   |   | klimatyzator.png
|   |   | layout/
|   |   |   | activity_main.xml
```

29.3 Projekt: Galeria — pełna lista plików

```
app/src/main/
|   | java/com/example/galeria/
|   |   | MainActivity.java
|   | res/
|   |   | drawable/
|   |   |   | oferta1.jpg      ← Zdjęcia przykładowe do galerii
|   |   |   | oferta2.jpg
|   |   |   | oferta3.jpg
|   |   |   | oferta4.jpg
|   |   | layout/
|   |   |   | activity_main.xml
```

29.4 Projekt: Notatki z wczytywaniem — pełna lista plików

```
app/src/main/
|   | assets/
|   |   | dane.txt            ← Plik z początkowymi notatkami
|   | java/com/example/notatki/
|   |   | MainActivity.java
|   | res/
|   |   | layout/
|   |   |   | activity_main.xml
```

30. Szybkie szablony — copy-paste gotowe fragmenty

30.1 Szablon: Inicjalizacja tablicy ImageView i obsługa kliknięć

```
// W MainActivity – tablica 5 ImageView
private ImageView[] ivKosci = new ImageView[5];

// W onCreate – inicjalizacja
int[] ids = {R.id.ivKosc1, R.id.ivKosc2, R.id.ivKosc3, R.id.ivKosc4, R.id.ivKosc5};
for (int i = 0; i < 5; i++) {
    ivKosci[i] = findViewById(ids[i]);
    final int idx = i;
    ivKosci[i].setOnClickListener(v → obsluzoKlikniecie(idx));
}
```

30.2 Szablon: Ustawianie obrazu kości po nazwie

```
private void ustawObrazKosci(ImageView iv, int liczbaOczek) {
    String nazwa = "kosc" + liczbaOczek;
    int resId = getResources().getIdentifier(nazwa, "drawable", getPackageName());
    if (resId ≠ 0) {
        iv.setImageResource(resId);
    } else {
        Log.e("TAG", "Brak zasobu: " + nazwa);
    }
}
```

Kotlin:

```
private fun ustawObrazKosci(iv: ImageView, liczbaOczek: Int) {
    val resId = resources.getIdentifier("kosc$liczbaOczek", "drawable", packageName)
    if (resId ≠ 0) iv.setImageResource(resId)
}
```

30.3 Szablon: Cykl elementów tablicy (poprzedni/następny)

```
private int aktualnyIndeks = 0;
private String[] elementy = {"A", "B", "C", "D"};

// Następny – z zawijaniem do początku
private void nastepny() {
    aktualnyIndeks = (aktualnyIndeks + 1) % elementy.length;
    aktualizuj();
}

// Poprzedni – z zawijaniem do końca
private void poprzedni() {
    aktualnyIndeks = (aktualnyIndeks - 1 + elementy.length) % elementy.length;
}
```

```
    aktualizuj();  
}
```

Kotlin:

```
var aktualnyIndeks = 0  
val elementy = arrayOf("A", "B", "C", "D")  
  
fun nastepny() { aktualnyIndeks = (aktualnyIndeks + 1) % elementy.size }  
fun poprzedni() { aktualnyIndeks = (aktualnyIndeks - 1 + elementy.size) % elementy.size }
```

30.4 Szablon: Walidacja pola liczbowego z zakresem

```
private int pobierzLiczbePelna(EditText et, int min, int max) {  
    String tekst = et.getText().toString().trim();  
    if (tekst.isEmpty()) return -1;  
    try {  
        int n = Integer.parseInt(tekst);  
        return (n ≥ min && n ≤ max) ? n : -1;  
    } catch (NumberFormatException e) {  
        return -1;  
    }  
}  
  
// Użycie  
int numer = pobierzLiczbePelna(etNumer, 1, 12);  
if (numer == -1) {  
    tvBlad.setText("Podaj liczbę z zakresu 1-12");  
    return;  
}
```

Kotlin:

```
fun pobierzLiczbe(et: EditText, min: Int, max: Int): Int? {  
    val tekst = et.text.toString().trim()  
    val n = tekst.toIntOrNull() ?: return null  
    return if (n in min..max) n else null  
}  
  
// Użycie  
val numer = pobierzLiczbe(etNumer, 1, 12) ?: run {  
    tvBlad.text = "Podaj liczbę z zakresu 1-12"  
    return  
}
```

30.5 Szablon: Wyświetlanie listy z ArrayAdapter

```
private ArrayList<String> lista = new ArrayList<>();  
private ArrayAdapter<String> adapter;
```



```

// W onCreate
lista.add("Element 1");
lista.add("Element 2");

adapter = new ArrayAdapter<(this, android.R.layout.simple_list_item_1, lista);
listView.setAdapter(adapter);

// Dodawanie
public void dodajElement(String el) {
    if (!el.isEmpty()) {
        lista.add(el);
        adapter.notifyDataSetChanged();
    }
}

```

30.6 Szablon: Odczyt z pliku assets z obsługą błędów

```

private ArrayList<String> wczytajPlik(String sciezka) {
    ArrayList<String> wynik = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(
        new InputStreamReader(getAssets().open(sciezka), "UTF-8"))) {
        String linia;
        while ((linia = br.readLine()) != null) {
            String czysta = linia.trim();
            if (!czysta.isEmpty()) wynik.add(czysta);
        }
    } catch (IOException e) {
        Log.e("IO", "Błąd wczytywania: " + sciezka, e);
        Toast.makeText(this, "Błąd odczytu: " + sciezka, Toast.LENGTH_SHORT).show();
    }
    return wynik;
}

```

Tabele szybkiego dostępu

Atrybuty XML — najczęstsze

ATRYBUT	OPIS	PRZYKŁAD WARTOŚCI
<code>android:id</code>	Identyfikator elementu	<code>@+id/btnOK</code>
<code>android:layout_width</code>	Szerokość	<code>match_parent</code> / <code>wrap_content</code> / <code>100dp</code>
<code>android:layout_height</code>	Wysokość	<code>match_parent</code> / <code>wrap_content</code> / <code>60dp</code>
<code>android:text</code>	Wyświetlany tekst	<code>"Kliknij"</code> / <code>@string/btn_ok</code>
<code>android:textSize</code>	Rozmiar tekstu	<code>16sp</code> / <code>24sp</code>
<code>android:textColor</code>	Kolor tekstu	<code>#FFFFFF</code> / <code>@color/white</code>
<code>android:textStyle</code>	Styl tekstu	<code>bold</code> / <code>italic</code> / <code>normal</code>
<code>android:background</code>	Tło	<code>#FF5733</code> / <code>@color/primary</code>
<code>android:backgroundTint</code>	Tint tła (przyciski)	<code>#D2691E</code>
<code>android:gravity</code>	Wyrównanie zawartości	<code>center</code> / <code>start</code> / <code>end</code>
<code>android:layout_gravity</code>	Wyrównanie w rodzicu	<code>center</code> / <code>center_horizontal</code>
<code>android:padding</code>	Odstęp wewnętrzny	<code>8dp</code> / <code>16dp</code>
<code>android:layout_margin</code>	Odstęp zewnętrzny	<code>10dp</code>
<code>android:orientation</code>	Kierunek (LinearLayout)	<code>vertical</code> / <code>horizontal</code>
<code>android:layout_weight</code>	Waga (LinearLayout)	<code>1</code> / <code>2</code>
<code>android:src</code>	Źródło obrazu	<code>@drawable/kosc1</code>
<code>android:scaleType</code>	Skalowanie obrazu	<code>fitCenter</code> / <code>centerCrop</code>
<code>android:hint</code>	Podpowiedź (EditText)	<code>"Wpisz e-mail"</code>
<code>android:inputType</code>	Typ klawiatury	<code>text</code> / <code>number</code> / <code>textPassword</code>
<code>android:max</code>	Maksimum (SeekBar)	<code>40</code> / <code>100</code>
<code>android:progress</code>	Wartość startowa (SeekBar)	<code>20</code>
<code>android:checked</code>	Zaznaczony? (CheckBox)	<code>true</code> / <code>false</code>
<code>android:visibility</code>	Widoczność	<code>visible</code> / <code>invisible</code> / <code>gone</code>
<code>android:divider</code>	Separator listy	<code>#DC143C</code>
<code>android:dividerHeight</code>	Grubość separatora	<code>1dp</code>

Typy inputType (EditText)

WARTOŚĆ	EFEKT
text	Zwykły tekst
textPassword	Maskowane znaki
textEmailAddress	Klawiatura z @ i .com
number	Tylko cyfry
numberSigned	Cyfry + minus
numberDecimal	Cyfry + przecinek
phone	Klawiatura numeryczna
textMultiLine	Wiele linii, Enter dozwolony
textCapSentences	Duże litery na początku zdań